

CAOS - Internal Development Readiness Package

CAOS

From Chaos to CAOS

Practice Automation Platform — Development Readiness Package **CONFIDENTIAL — INTERNAL REVIEW ONLY**

Prepared for internal review. Not for external or client distribution — several sections (architecture decisions, security posture, API surface, coding standards) are not appropriate to share outside the delivery team. The externally shareable companion document is *CAOS — Feature & Process Documentation*.

Version History

Version	Date	Summary
v0.1	2026-08-16	First consolidated internal review package. Combines the PRD, Feature Backlog, 10 Architecture Decision Records, ER diagrams, API specification, wireframes, visual design directions, Coding Guidelines, Logging Standard, Testing Strategy, Security Standard, Performance & Scaling targets, and the Sprint Plan into a single reviewable document.

Contents

Full navigation is available via this PDF's bookmarks/outline panel (every heading in every section is bookmarked). At the top level:

1. Product Requirements Document (PRD)
2. Feature Backlog
3. Architecture Decision Records (10 ADRs)
4. ER Diagram
5. API Specification

6. Wireframes
 7. Visual Design Directions
 8. Coding Guidelines
 9. Logging Standard
 10. Testing Strategy
 11. Security Standard
 12. Performance & Scaling
 13. Sprint Plan
-

What CAOS Is

CAOS is a practice-automation platform being built for Venture Assist / Srivatsan & Associates — a CA practice currently servicing roughly 100 GST returns per month with a goal of reaching 500 within 6 months, without adding headcount. The name plays on the problem it solves: a set of manual, fragmented processes (document chasing, bookkeeping, GST reconciliation, client follow-up, billing) that today depend on individual discipline and don't scale — turned into one coherent, task-driven operating system for the practice.

CAOS is not a single application in the conventional sense. It's a task-engine-driven platform with a set of purpose-built agents — document intake, bookkeeping, GST/TDS reconciliation, compliance validation, client communication, billing — all writing into one shared Task Engine, so that every human judgment call becomes an owned task with a due date, not a notification that can be missed. It's an internal operations tool, not a client-facing product; the practice's client acquisition stays personal-connect only, by design.

What This Document Is

This is the complete internal development-readiness package: everything decided, documented, and specified before writing application code, consolidated for review by whoever needs visibility into the full technical and product picture — not just the piece they're directly responsible for. It mirrors the structure of a prior reference project's own readiness documentation, adapted to this project's actual scope and decisions rather than copied from it.

Read end-to-end for a full picture, or jump to the section relevant to your review using the table of contents.

Practice Automation Platform — Product Requirements Document

Prepared for Venture Assist / Srivatsan & Associates | Draft v0.1

Changelog

Version	Date	Summary
v0.1	2026-08-16	Initial draft: 13 modules, requirements, user stories, KPIs. Open questions resolved iteratively: Tally access method, Winman/GSP integration approach, Dropbox API constraints, DPDP/data-residency architecture principle, billing module added. Still open: GSP vendor pricing confirmation, DPDP specifics. Not yet at v1.0 — pending ADRs.

1. Executive Summary

This PRD defines the requirements for an internal practice-automation platform: a task-engine-driven system with a set of purpose-built agents that automate document intake, bookkeeping, GST/TDS reconciliation, compliance validation, client communication, and practice-wide workload visibility for a CA practice currently servicing ~100 GST returns/month with a target of ~500/month within 6 months, without adding headcount.

This is an **internal operations platform**, not a client-facing product — client acquisition stays personal-connect only, per the practice’s stated preference. Every module exists to remove manual, repetitive work from the 10-person team while keeping the proprietor and senior staff in control of every judgment call.

2. Platform at a Glance

Dimension	Detail
Platform type	Internal task-engine + automation-agent platform for a CA/audit practice
App form factor	PWA (installable) and regular full-browser access — same app, no separate desktop-only build. Designed desktop-first with responsive breakpoints, since staff primarily work at a desktop monitor; mobile is a secondary, fully-functional but reflowed layout, not the

Dimension	Detail
Primary users	primary design target Proprietor, senior staff (paid assistants), junior staff (article clerks)
External users	None — no client-facing portal in this phase
Data sources	Tally Cloud (books), Dropbox (documents), shared inbox work@xyz.com (email), Winman (filing utility), GSTN portal (GSTR- 2A/2B)
Geography / compliance	India — GST, Income Tax, TDS/TCS, ROC
Language	English (staff-facing; no multi-language requirement identified)
Architecture	Not yet decided — see Open Questions (§8) and next-step Architecture Decisions
Modules	14 (13 functional + 1 integration layer)
Build plan	Phased (Phase 0–4, per prior roadmap discussion) — to be broken into a sprint plan once this PRD and the architecture decisions are locked

3. Documentation Roadmap

Tracking status of the full readiness package — this table is the master index across all artifacts, so this stays the single reference point as the rest gets built. Update the Version and Last Updated columns whenever a document changes.

Document	Status	Version	Last Updated
PRD (this document)	In progress	v0.1	2026-08-16
Feature Backlog (MoSCoW + phase mapping)	Complete (102 features, xlsx with Summary sheet)	v0.1	2026-08-16
Architecture Decision Records	In progress (10 ADRs — 9 accepted, 1 proposed; DPDP legal specifics, IaC tooling, and extension-point pattern still open)	v0.1	2026-08-16
ER Diagram	Complete (2 diagrams: core platform, compliance & billing — 23 entities)	v0.1	2026-08-16

Document	Status	Version	Last Updated
API Spec	Complete — 58 endpoints across 12 domains, traced to requirement IDs; design-time reference, not the final FastAPI-generated OpenAPI artifact	v0.1	2026-08-16
Coding Guidelines	Complete — CG1–CG11, resolves ADR 0010’s extension-point pattern	v0.1	2026-08-16
Logging Standard	Complete — correlation fields, log levels, audit-trail boundary, PII masking	v0.1	2026-08-16
Testing Strategy	Complete — test pyramid, mocking approach, 4 critical E2E flows, CI gates	v0.1	2026-08-16
Security Standard	Complete — auth/authz, secrets, encryption, audit-trail, GSP rate limiting	v0.1	2026-08-16
Performance & Scaling	Complete — per-deployment targets, scaling triggers (not multi-tenant capacity planning)	v0.1	2026-08-16
Wireframes / Mockups	Complete — Design B (Capacity) applied across all 9 screens + 4 Admin sub-screens	v0.2	2026-08-16
Visual Design Directions	Resolved — customer selected Design B (Capacity): teal/amber, Inter, radial gauge signature element	v0.1	2026-08-16
Tech Stack Decision	Drafted as ADR 0007	v0.1	2026-08-16

Document	Status	Version	Last Updated
	(Proposed — Python/FastAPI backend, hosting revised to AWS Mumbai for DPDP alignment)		
Sprint Plan	Complete — Phase 0 + 12 sprints (~27 weeks), sequenced against the Feature Backlog's phase mapping and every ADR's dependency order	v0.1	2026-08-16
Dev Readiness Checklist	Not started	—	—

4. Users & Roles

Role	Who	Primary use of the platform
Proprietor (SV)	Practice owner	Cross-client/cross-staff dashboard, escalations, final sign-off on judgment calls, capacity view for taking on new work
Senior staff (paid assistants)	6	Reconciliation/TDS/validation review, voucher approval, client communication approval, task assignment to juniors
Junior staff (article clerks)	4	Document classification exceptions, staged voucher review, routine task execution, follow-up calls

Note: all three roles were confirmed as day-to-day users during discovery (Q19) — this isn't a tool for one persona, it's used practice-wide.

5. Modules & Requirements

5.0 Architecture Principle: Platform as a Thin Layer, Not a New System of Record

Tally (via the Connector) and Dropbox remain the systems of record for financial data and documents respectively — both are already in use today, and both already carry whatever data-handling posture the practice and its clients have accepted. The platform does **not** introduce a new consolidated store of full financial records or documents.

What the platform's own database *does* hold: task metadata, client profile/billing data, communication logs, and reconciliation/validation results (derived data, not source documents). Source documents and ledger data are read from Dropbox/Tally on demand, processed, and written back — not duplicated into a separate permanent store.

This narrows the DPDP Act surface considerably (§8, item 4): the practice's existing obligations for Tally and Dropbox already cover the bulk of the data; what's new is the platform's own metadata layer, plus any transient processing (OCR/LLM) in between — which is a smaller, more specific question than “where does all client data live now.”

5.1 Identity & Access (ID)

Role-based access and accountability across the practice.

- **ID-01** Role-based login (Proprietor / Senior staff / Junior staff)
- **ID-02** Each user sees a personal task queue scoped to what's assigned to them
- **ID-03** Proprietor has cross-client, cross-staff visibility by default
- **ID-04** Session handling appropriate for shared office devices (auto-logout, no persistent shared sessions)
- **ID-05** Audit log of who took what action, when — supports accountability and DPDP Act alignment

5.2 Task Engine (TE)

The shared layer every other module writes to and every user reads from. This is the platform's core — see the earlier discussion on making every human touchpoint a routed, owned task.

- **TE-01** Any agent exception or judgment call creates a task record: type, linked record/context, client, due date
- **TE-02** Task routing via a configurable assignment rule table (task type — default role/person)
- **TE-03** Manual reassignment available to senior staff and proprietor
- **TE-04** Auto-escalation when a task passes its due date/threshold — escalates to senior staff or proprietor depending on task type
- **TE-05** Task status lifecycle: Open — In Progress — Completed / Escalated
- **TE-06** Completing a task requires a recorded outcome (approved / rejected / edited), not just “marked done”
- **TE-07** Personal “My Tasks” view, sorted by due date and priority
- **TE-08** Notification on new assignment and on escalation

- **TE-09** Full task history retained per client, for audit trail and time-per-task measurement

5.3 Tally Connector (TC) — integration layer, not user-facing

Everything else depends on this working reliably. Confirmed: books for the large majority of clients are maintained by the practice's own staff in the practice's own Tally Cloud (multi-company subscription) — this is the practice's environment, not each client's. Only a small percentage of clients maintain their own Tally and hand over a backup/exported ledger instead. This means two distinct data paths, not one.

Primary path — practice-maintained companies (majority of clients) - **TC-01** Connect to the practice's own Tally Cloud subscription, across all client companies hosted within it, via TallyPrime's built-in XML-over-HTTP interface (default port 9000). **Verified against Tally Solutions' own documentation:** this is a native, built-in capability of TallyPrime — enabling it is a configuration step (Advanced Configuration → HTTP Server), not a separate purchasable license. TallyPrime can act as an HTTP server, accepting XML/JSON requests and returning both read data and accepting writes (voucher/master creation) over the same interface. **Correction to the earlier assumption:** the real open item isn't licensing — it's whether the practice's Tally Cloud provider allows this port/configuration to be reached from outside their hosted environment. Worth re-confirming with the provider using this more precise framing. - **TC-01a** ODBC is also available on TallyPrime (as both ODBC server and client) but is **read-only** — no INSERT/UPDATE support. Suitable for simple reporting-style extraction if useful as a secondary/reporting path, but the Bookkeeping Agent's voucher push (TC-04) must go through the XML/HTTP interface, not ODBC. - **TC-01b** Development/testing note: Tally Solutions provides an official sandbox — the **TallyPrime API Explorer** — for sending live XML/JSON requests and inspecting responses without a real Tally company connected. Use this to validate the exact payload shapes for TC-02 (extract) and TC-04 (voucher push) before writing Connector code against any live client data — directly reduces risk against the DPDP concern raised earlier (\$5.0), since development doesn't need production client data at all. The **TallyPrime Developer IDE** (includes a Tally Connector component) is the tool for the actual build; it has a 90-day free evaluation, after which a Tally Software Services (TSS) subscription is needed — small recurring cost worth including in the investment conversation. - **TC-02** Extract purchase register, sales register, bank book, and ledgers in structured form, per client company - **TC-03** Normalize data across Tally versions/company-specific customizations into one standard schema - **TC-04** Push draft vouchers (from the Bookkeeping Agent) into the correct client company via import - **TC-05** Per-client-company connection/sync health surfaced on the Practice Management dashboard

Secondary path — client-maintained books (small percentage of clients) - **TC-06** Accept a client-provided Tally backup file as the input — confirmed this is the actual format used, and it can be restored/imported directly into the practice's own Tally environment - **TC-07** Once restored, the same primary-path extraction (TC-01-TC-03) applies — no separate parser needed, just an operational step (restore backup → company appears in Tally → primary pipeline runs against it) with its own scheduling/freshness profile - **TC-08** Flag clients on this path distinctly on the dashboard, since data freshness depends on how often the client sends an updated backup — this is a materially different reliability profile than the live-connected majority

5.4 Email Intake Agent (EI)

Automates the manual email-to-Dropbox filing step confirmed as a real, unautomated pain point.

- **EI-01** Monitor the shared inbox (work@xyz.com) for incoming client emails
- **EI-02** Identify the client from sender address/domain, using a learned mapping table
- **EI-03** Classify the attachment type (invoice, bank statement, GSTR filing, other)
- **EI-04** High-confidence match → auto-file to the correct Dropbox folder
- **EI-05** Low-confidence or unrecognized sender → creates a task for staff to confirm client/folder manually
- **EI-06** Sender-to-client mapping is viewable and editable by staff

5.5 Document Intake / Dropbox Sync (DI)

Verified against Dropbox's own API documentation. Confirmed capabilities and design constraints below.

- **DI-01** Watch the defined Dropbox folder structure (client/period/doc-type) via webhook. **Verified:** Dropbox webhooks notify only that an account has changes — not which files. The confirmed pattern is: webhook fires → app calls /files/list_folder/continue with a stored cursor → app filters the returned changes for relevant folders. This two-step flow is a required design detail, not an implementation nicety.
- **DI-01a Hard constraint:** the webhook endpoint must respond within 10 seconds. Actual processing (fetching changes, downloading the file, triggering OCR) must happen asynchronously via a queue/background worker — never inline in the webhook handler.
- **DI-01b Recommended scope decision:** request Dropbox “App folder” access rather than “Full Dropbox” access. Since the practice’s Dropbox holds the entire company’s client data, scoping the platform’s access to its own designated folder tree is a concrete, easy step toward the data-minimization principle already established (§5.0).
- **DI-02** OCR + data extraction for PDF documents (amounts, dates, GSTIN, vendor name)
- **DI-03** Excel parsing with per-client template detection, since formats vary by client
- **DI-04** Checklist of expected vs. received documents per client per period
- **DI-05** Missing-item reminder routed through the Client Communication Agent once a document is overdue

5.6 Bookkeeping / Voucher Entry Agent (BK)

Flagged directly by the customer as the area with the most automation need — this is the highest-leverage module. Confirmed this is the primary flow for the large majority of clients, since the practice enters books into its own Tally Cloud from client-supplied source documents; only a small percentage of clients maintain their own books and skip this agent entirely (see TC secondary path, §5.3).

- **BK-01** Extract line items from intake documents: vendor, date, amount, GST, HSN

- **BK-02** Fuzzy-match vendor/customer name to an existing Tally ledger
- **BK-03** No match found → create a task for new-ledger creation rather than auto-creating
- **BK-04** Generate draft Tally voucher(s) with correct CGST/SGST/IGST ledger split
- **BK-05** Confidence scoring: high-confidence vouchers auto-stage, low-confidence ones are flagged for review
- **BK-06** Staff review/approval screen for staged vouchers before posting
- **BK-07** Approved vouchers post to Tally via the Connector
- **BK-08** Duplicate-prevention check runs before posting — verified that Tally’s XML import does not prevent duplicate vouchers on its own, so the platform must check for an existing voucher (matching vendor GSTIN + invoice number + date) before posting, not rely on Tally to catch it

5.7 Reconciliation Agent (RC)

- **RC-01** Pull purchase register (via Tally Connector) and GSTR-2A/2B via a direct GSP (GST Suvidha Provider) API integration — **this replaces the Winman-dependent path entirely**. Winman’s export capability confirmed the data *can* leave Winman, but doing so still requires a human to operate Winman’s desktop UI per client per period, which is exactly the bottleneck we’re trying to remove. GSTN licenses GSPs to provide programmatic API access to the same underlying data Winman itself uses internally — registering the practice directly with a GSP removes Winman from the reconciliation critical path altogether. Shortlist of GSTN-licensed GSPs with developer-facing APIs worth evaluating: WhiteBooks (has a free sandbox and a purpose-built GSTR-2B fetch/reconciliation API — good first candidate to prototype against), GSTHero, Adaequare, GSPIndia, MasterGST, Cygnet, IRIS GST.
- **RC-01a Note on what doesn’t go away:** GSTN requires an OTP/consent step per client GSTIN as part of its own security model, regardless of which GSP is used. This becomes a scriptable, automated authentication step within the platform rather than a manual per-client action in Winman — the bottleneck being removed is the manual UI operation, not the GSTN-mandated consent step itself.
- **RC-01b** GSP API access has its own cost (typically per-call or subscription-tier pricing) — a new line item to include in the investment conversation, similar to the Tally XML/ODBC licensing note (\$5.3)
- **RC-01c Strategic scope note — worth reading carefully before assuming “replace Winman”:** Winman covers three distinct compliance areas (GST, Income Tax/ITR, TDS), and they don’t have equivalent replacement paths. GST filing and reconciliation have solid API-first alternatives (WhiteBooks, ClearTax GST API) — the same GSP relationship used for GSTR-2A/2B (RC-01) can likely also handle GSTR-1/3B filing itself, meaningfully reducing Winman’s role in the GST workflow specifically. Income Tax (ITR) and TDS filing run through *separate* government systems (Income Tax Department e-filing infrastructure, TRACES) — replacing Winman there would need its own integration effort (e.g., ERI — e-Return Intermediary — registration), not a simple vendor swap. **Recommendation:** scope this build to replacing Winman’s GST role only, for now — that’s what’s actually blocking the 500-returns target. Full Winman replacement across ITR/TDS is a separate future initiative, not part of this phase.

- **RC-01d WhiteBooks technical spike details (verified, safe to act on):** sandbox at `apisandbox.whitebooks.in/gst` (free, no card required), production at `api.whitebooks.in/gst`, OAuth 2.0 bearer auth, relevant endpoints for GSTR-1/GSTR-3B filing and GSTR-2B fetch, SDKs in Java/Node.js/Python, OpenAPI 3.0 spec available. **Caveat:** WhiteBooks' own pricing page shows internally inconsistent numbers (fixed tiers in one section, "contact sales for custom pricing" in another) alongside an enterprise client list that reads as inflated marketing. Treat the sandbox/technical details as safe to prototype against immediately; do not use any pricing figures from their site in the investment conversation without a direct sales quote.
- **RC-02** Match line items by GSTIN, invoice number, and amount
- **RC-03** Auto-confirm matched items and include in the ITC claim summary
- **RC-04** Mismatches (timing difference, vendor not filed, amount mismatch, duplicate) become a task with reason
- **RC-05** Generate the final reconciliation statement for filing

5.8 Validation / Compliance Agent (VC)

Verified: Winman is a desktop application with no open third-party API. GSTR-2A/2B data now flows through a direct GSP integration instead (\$5.7, RC-01), bypassing Winman entirely for that step. Winman remains relevant for filing itself: it supports Excel export, and import via JSON or Excel — but only per individual head of income, not as a holistic single import. This means any automated pre-fill back into Winman for filing purposes has to be built head-by-head, with a review step per head, rather than one bulk push.

- **VC-01** GSTR-1 output tax vs. GSTR-3B output tax cross-check
- **VC-02** HSN code presence and correctness check
- **VC-03** Tax rate applied vs. standard rate table
- **VC-04** TDS in Form 26AS vs. books cross-check
- **VC-05** Any discrepancy becomes a task with the specific line item and variance amount, ahead of the filing deadline
- **VC-06** Per-client filing-readiness status (cleared / blocked)

5.9 Client Communication Agent (CC)

Addresses the #1 stated pain point — follow-up.

- **CC-01** Template library for routine reminders (deadline approaching, document pending)
- **CC-02** Routine reminders sent automatically on a configured schedule
- **CC-03** Communication requiring judgment is drafted and routed as a task for approval before sending
- **CC-04** All sent/received communication logged per client
- **CC-05** Time-sensitive, client-dependent actions (e.g. OTP-based steps) flagged with higher priority — directly addresses the discovery finding that staff need to know *when* a client should be called, not just that they should be

5.10 TDS/TCS Return Automation (TDS)

- **TDS-01** Pull TDS deducted (books) and Form 26AS/TRACES data
- **TDS-02** Match deductions against deposits and due dates
- **TDS-03** Short deduction, late deposit, or mismatch becomes a task with the specific entry
- **TDS-04** Prepare the draft quarterly return (24Q/26Q/27EQ) for review and filing

5.11 Bank Reconciliation Agent (BR)

- **BR-01** Pull bank statement data and Tally bank book, across all client accounts
- **BR-02** Auto-match entries by date, amount, and reference
- **BR-03** Unmatched entries become a task for staff review
- **BR-04** Generate a reconciliation statement per account, per period

5.12 Working Paper Agent (WP)

- **WP-01** Auto-populate the standard working paper template from reconciliation and validation output
- **WP-02** Generate a draft working paper per engagement
- **WP-03** Task routed to the auditor to add professional judgment notes
- **WP-04** Finalized working paper filed to the client's Dropbox record

5.13 Client Profiling & Billing (CB)

Confirmed: billing today is informal — based on individual job description and client, done ad hoc in Excel, with no existing tier or rate-card structure. This module is being designed from scratch, not digitizing an existing system. Invoices must be GST-compliant (GSTIN, HSN/SAC codes for professional services, CGST/SGST/IGST breakup).

- **CB-01** Client master profile: entity type, services subscribed (GST / IT / Statutory Audit / ROC / Advisory), linked Tally company, primary contact, billing cycle (monthly/quarterly)
- **CB-02** Service catalog: each billable service type defined with standard price tiers (e.g. by client size/complexity/return volume) — built from scratch, since no rate card exists today. Category structure drawn from the practice's own two websites (Venture Assist, Srivatsan & Associates), which between them list a broader service line than initially scoped: GST (filing, reconciliation, registration/migration, notice representation), Income Tax (filing, notice representation), Accounting Services (bookkeeping, financial reporting/MIS, budgeting), Statutory Audit, Entity Setup & Registration (incorporation, trademark, PF/ESI), Corporate/ROC Compliance (annual compliance, governance, NCLT advisory), Corporate Finance & Advisory, and Services for Non-Residents. Not all of these are automation targets — the catalog exists to support billing across the practice's full service line, whether or not a given service has an agent behind it yet.
- **CB-03** Per-client price override — a specific client's rate can be set to override the tier default, with the reason and who approved it recorded

- **CB-04** Billing-trigger mapping: which completed agent/task event generates a billable line item (e.g. a completed GST filing — filing fee; bookkeeping voucher volume in a period — bookkeeping fee at the applicable tier)
- **CB-05** Auto-generate a draft invoice per client per billing cycle, built from services actually rendered and the applicable tier/override pricing
- **CB-06** Draft invoice routed as a task for proprietor/senior staff review and approval before it's sent — consistent with the platform-wide principle that anything requiring judgment becomes an owned task, not an auto-send
- **CB-07** Invoice history per client: sent, outstanding, paid status
- **CB-09** Invoice format is GST-compliant: GSTIN, HSN/SAC code for professional services, and CGST/SGST/IGST tax breakup on every generated invoice
- **CB-08** Tier reassignment workflow for when a client's volume or complexity changes (e.g. crosses a return-count threshold) — flagged as a task rather than silently auto-changing a client's pricing

5.14 Practice Management Dashboard (PM)

- **PM-01** Cross-client, cross-staff task dashboard for the proprietor
 - **PM-02** Per-staff workload view: open, overdue, and completed tasks
 - **PM-03** Capacity indicator — surfaces available bandwidth for new client work
 - **PM-04** Deadline calendar across all clients and all compliance types (GST, IT, TDS, ROC)
 - **PM-05** Time-per-task tracking — extends the practice's existing task-sheet habit rather than replacing it outright, and supplies the real numbers needed for ongoing ROI measurement
 - **PM-06** Outstanding/overdue invoice visibility, sourced from the Client Profiling & Billing module (§5.13)
-

6. Deferred to Later Phases (Out of Scope for This PRD)

Confirmed low-priority or explicitly deprioritized during discovery:

- **Notice / Scrutiny Tracker Agent** — notices are “negligible” per discovery; revisit if that changes
 - **AI-assisted audit tooling** — explicitly flagged as not relevant at current client scale
 - **Client-facing portal** — client acquisition is personal-connect only; no external-facing surface planned
 - **Financial Statement Analysis/Ratio Agent, ROC Tracker, Fixed Asset Register, Audit Sampling & Risk-Scoring, Knowledge Base/Query Assistant** — all previously identified as potential value-adds, not core to the 500-returns goal
-

7. Key User Stories

- As a **junior staff member**, I want one queue of tasks assigned to me with due dates, so I know exactly what to work on without checking multiple channels.

- As the **proprietor**, I want to see which clients are at risk of missing a filing deadline, so I can step in before it becomes urgent.
 - As a **senior staff member**, I want to review only the flagged reconciliation mismatches instead of the full ledger, so I save review time on every client.
 - As a **staff member**, I want client documents to be filed to the right Dropbox folder automatically, so I stop spending time on manual email filing.
 - As the **proprietor**, I want an overdue task to escalate to me automatically, so a missed internal deadline doesn't go unnoticed the way it can today.
 - As the **proprietor**, I want invoices to be drafted automatically from services actually delivered, so billing doesn't depend on someone remembering to work it out manually each cycle.
-

8. Open Questions (Before Architecture Decisions)

Answers received from the client's licensing consultant — most are resolved, with a couple of technical follow-ups still needed before the Tally Connector ADR can be finalized.

#	Question	Answer	Status
1	Tally Cloud access method	Browser-based access, from anywhere, with username/password. Verified: TallyPrime's XML-over-HTTP interface (port 9000) and ODBC are both built-in, native capabilities — not separate licenses. The real question is whether the Tally Cloud provider's hosting allows this port/config to be reached externally	Resolved (technical approach confirmed), one item to reconfirm — re-ask the provider specifically: "can the HTTP server on port 9000 be reached from outside your hosted environment, or can a small service be deployed on the same host to reach it locally?" This is a hosting/network question now, not a licensing one
1a	Format for the small percentage of clients on their own Tally	Native Tally backup file, importable directly into Tally	Resolved — simplifies the secondary path considerably (\$5.3, TC-06/07)
2	Does Winman have export/API capability?	Yes — exports to Excel; import via JSON or Excel, but only per individual	Resolved — see VC module note (\$5.8)

#	Question	Answer	Status
3	How is GSTR-2A/2B data accessed?	head of income, not holistically Client said “through the Winman API,” which turned out to describe Winman’s internal GSP connectivity, not a callable API. Winman does support file export, but that still requires manual UI operation per client — the actual bottleneck	Resolved (new approach) — bypass Winman for this data pull; register the practice directly with a GSTN-licensed GSP (shortlist in RC-01, \$5.7) for programmatic, multi-client API access. Removes the manual bottleneck at its root rather than working around it
4	DPDP Act obligations for storing/processing client financial data	Tally and Dropbox are the systems of record and already carry the practice’s existing data-handling posture; the platform doesn’t introduce a new consolidated store — see the Architecture Principle at the start of §5	Largely resolved — narrowed to: what DPDP obligations apply specifically to the platform’s own metadata layer (tasks, billing, communication logs) and to transient OCR/LLM processing (still item 5)
5	Where does OCR/LLM processing run, and what data leaves the practice’s environment?	Documents stay in Dropbox as the system of record (per the Architecture Principle above), which resolves <i>where data lives</i>. What’s still open is narrower: which OCR/LLM vendor processes documents in transit, and whether that vendor retains any data even transiently	Partially resolved — storage question closed; vendor/processing-transit question remains, but it’s a smaller, more specific decision than before
6	What does client	Informal/ad hoc, per	Resolved — Billing

#	Question	Answer	Status
	billing look like today?	job and per client, currently done in Excel; invoices must be GST-compliant	module (\$5.13) designed from scratch, GST-compliance requirement added (CB-09)

9. Success Metrics (KPIs)

Metric	Today	6-Month Target
GST returns serviced/month	~100	~500
Reconciliation time per client	1–2 hrs/month	<15 min (review only)
Tasks completed before due date	Not tracked	Target: >90%
Documents auto-filed without manual intervention	0% (fully manual)	Majority, exceptions only
Vouchers auto-staged with high confidence	0%	Target set after Phase 0 baseline
Follow-up handled without manual chasing	0% (fully manual, #1 pain point)	Majority automated, exceptions only
Invoices auto-drafted from rendered services	0% (manual/ad hoc)	Majority auto-drafted, exceptions reviewed only

10. Next Steps

1. Resolve the open questions in §8 that block architecture decisions — Tally Cloud access method is the most urgent, since it affects the Connector design that most other modules depend on.
2. Write the Architecture Decision Records covering: Tally Connector approach, task engine data model, offline/PWA sync strategy, and OCR/document-processing approach.
3. Convert this PRD into a feature backlog with MoSCoW prioritization and phase mapping (Phase 0–4).
4. Draft the ER diagram once the data model is settled.
5. Sequence the sprint plan against the phased roadmap already agreed with the customer.

Feature Backlog

MoSCoW Prioritization + Phase Mapping

The full feature backlog (102 features, MoSCoW-tagged, mapped to Phase 0 through Phase 4 and Future) is maintained as a separate spreadsheet for filtering/sorting: `audit-platform-feature-backlog-v0.1.xlsx`, distributed alongside this document.

Summary — by phase:

Phase	Count
Phase 0 (Baseline)	5
Phase 1 (Foundation)	27
Phase 2 (Bookkeeping + Communication)	21
Phase 3 (Reconciliation + Validation + TDS)	15
Phase 4 (Remaining agents + Billing)	23
Future (deferred)	11

Summary — by MoSCoW category:

Category	Count
Must	71
Should	18
Could	2
Won't (this phase)	11

See the spreadsheet's `Feature Backlog` tab for the full itemized list with descriptions and traceability, and the `Summary` tab for these counts as live formulas.

Architecture Decision Records

Practice Automation Platform — Venture Assist / Srivatsan & Associates

Changelog

Version	Date	Summary
v0.1	2026-08-16	Initial set: ADR 0001–0006, formalizing decisions established during PRD research (Tally integration, GST data access, Dropbox intake, data architecture, task engine, app form factor). ADR 0007 (tech stack) added, then revised: backend changed from NestJS to Python/FastAPI on recognizing the document-extraction/matching engine as the differentiator. ADR 0008 added: self-hosted PaddleOCR (PP-StructureV3) chosen for DPDP-driven data residency, resolving the OCR vendor open item. This surfaced a hosting conflict — Railway has no India region — so ADR 0007’s hosting choice was revised to AWS ap-south-1 (Mumbai). ADR 0009 added: single-tenant, per-customer deployment model confirmed (not multi-tenant) — no data-model change needed. ADR 0010 added: base product + per-practice customization + upstream promotion model, resolving ADR 0009’s update/patch strategy open item. DPDP legal specifics still open.

Each ADR follows: Status, Context, Decision, Consequences, Alternatives Considered. Status is one of: Proposed, Accepted, Superseded.

ADR 0001 — Tally Integration via XML-over-HTTP

Status: Accepted

Context: The platform needs both read access (purchase register, ledgers, bank book) and write access (posting draft vouchers) to the practice's Tally Cloud environment. Initial assumption was that this required procured licensing; verified against Tally Solutions' own documentation that this is a native, built-in capability.

Decision: Use TallyPrime's XML-over-HTTP interface (default port 9000) as the primary integration method, for both reads and writes. ODBC is available but read-only (no INSERT/UPDATE) — not usable for voucher posting, though it remains an option for simple reporting-style extraction if useful later. Development and testing happens against the official TallyPrime API Explorer sandbox before any real client data is touched.

Consequences: - The Tally Connector (a co-located service or reachable endpoint) must be able to reach port 9000 on the practice's Tally Cloud host — this is a hosting/network question to confirm with the provider, not a licensing cost. - Tally's XML import does not prevent duplicate vouchers on its own; the platform must implement its own duplicate check (vendor GSTIN + invoice number + date) before posting. - A TDS/TCS-style deep customization (via TDL) may be needed to shape reports/collections exactly as the platform needs them — this is development effort, not just configuration.

Alternatives considered: - *Browser automation against a hosted remote-UI* — rejected as the fallback only if XML/HTTP access turns out to be unreachable; less reliable, higher maintenance, avoided as the primary path. - *ODBC as primary* — rejected since it cannot write vouchers, which the Bookkeeping Agent requires.

ADR 0002 — GST Data Access via Direct GSP Integration (Bypassing Winman)

Status: Accepted

Context: GSTR-2A/2B reconciliation data was assumed to be accessible “through the Winman API.” Independent verification found Winman has no open third-party API — it's a desktop application that connects to the GST portal internally via its own GSP (GST Suvidha Provider) relationship. Having Winman export data still requires manual UI operation per client per period, which is the actual bottleneck the platform is meant to remove.

Decision: Register the practice directly with a GSTN-licensed GSP offering a developer REST API (shortlist: WhiteBooks, GSTHero, Adaequare, GSPIndia, MasterGST, Cygnet, IRIS GST), and integrate against it directly for both GSTR-2A/2B fetch and GSTR-1/3B filing. This removes Winman from the GST reconciliation/filing critical path entirely. Scope is limited to GST — Income Tax (ITR) and TDS filing remain on Winman for now, since they run through separate government systems (Income Tax e-filing infrastructure, TRACES) with their own integration requirements, and are not part of the current 500-returns-per-month goal.

Consequences: - GSTN's OTP/consent step per client GSTIN still applies regardless of GSP chosen — this becomes a scriptable, automated step rather than a manual one, but it doesn't disappear entirely. - GSP API access has its own subscription cost — a new line item for the investment conversation. - WhiteBooks' free sandbox (`apisandbox.whitebooks.in`) is confirmed safe to prototype against; their published pricing is not reliable (internally inconsistent on their own site) and needs a direct sales quote before being used in cost planning. - Full Winman replacement (ITR, TDS) is explicitly out of scope for this phase — noted as a candidate for a future initiative.

Alternatives considered: - *File-based export from Winman* — rejected as the primary path since it still requires a human to operate Winman's UI per client, which doesn't solve the bottleneck. - *Building an independent, non-GSP GSTN integration* — rejected; becoming a GSP directly is a heavy compliance undertaking disproportionate to this project's scope. Using an existing licensed GSP is the right level of effort.

ADR 0003 — Document Intake via Dropbox Webhooks with Async Processing

Status: Accepted

Context: Client documents arrive by email, are (currently manually) filed into Dropbox, and need to trigger downstream processing (OCR, classification, bookkeeping). Dropbox's webhook model was verified against their own API documentation.

Decision: Use Dropbox webhooks to detect account-level changes, followed by `/files/list_folder/continue` with a stored cursor to identify the specific changed files. The webhook handler acknowledges within Dropbox's 10-second response window and hands off actual processing (fetch, download, OCR trigger) to a background queue — never processed inline. Request "App folder" scope for the platform's Dropbox API app, rather than "Full Dropbox" access, scoping the platform to its own designated folder tree given the practice's Dropbox holds the entire company's client data.

Consequences: - Requires a queue/worker architecture from the start, not just a webhook endpoint — this is a concrete infrastructure requirement, not an optional enhancement. - App folder scoping is a meaningful, low-cost step toward the data-minimization principle established in ADR 0005. - The Email Intake Agent (which files documents into Dropbox in the first place) and the Document Intake Agent (which watches Dropbox for changes) are two distinct agents with a clean handoff point — worth keeping that separation clear in the eventual ER diagram and API spec.

Alternatives considered: - *Polling `/files/list_folder` on a schedule* — rejected as inefficient and higher-latency compared to webhooks; Dropbox's own guidance discourages rapid polling for server-side apps. - *Processing synchronously inside the webhook handler* — rejected; violates the 10-second response constraint for anything beyond trivial payloads.

ADR 0004 — Platform as a Thin Layer Over Tally and Dropbox

Status: Accepted

Context: DPDP Act obligations and data-residency questions were raised as open items. Tally and Dropbox are already the practice's systems of record and already carry whatever data-handling posture the practice and its clients have accepted.

Decision: The platform does not introduce a new consolidated store of full financial records or documents. Tally (via the Connector) and Dropbox remain the systems of record. The platform's own database holds only derived/operational data: task metadata, client profile and billing data, communication logs, and reconciliation/validation results. Source documents and ledger data are read on demand and written back, not duplicated into a separate permanent store.

Consequences: - Narrows the DPDP Act surface considerably — most obligations are already covered by the practice's existing use of Tally and Dropbox; what's new is the platform's own metadata layer plus transient OCR/LLM processing. - Still requires a decision on which OCR/LLM vendor processes documents in transit, and whether that vendor retains anything even temporarily — this is a smaller, specific decision rather than a broad open question. - Any future feature that seems to want a "local copy" of source documents or full ledgers should be treated as a deviation from this ADR and revisited explicitly, not built by default.

Alternatives considered: - *Full data warehouse mirroring Tally + Dropbox into the platform's own store* — rejected; unnecessary duplication, expands the DPDP surface significantly, and adds sync-consistency risk for no clear benefit given Tally/Dropbox already serve as reliable sources of truth.

ADR 0005 — Task Engine as the Shared Backbone

Status: Accepted

Context: Customer feedback was explicit: any human-touch step across any agent must become a task assigned to a specific person with a due date — not a generic notification or flag. Internal deadlines are currently missed due to inconsistent follow-through (a discipline gap), not tool resistance.

Decision: Every agent-raised exception or judgment call writes to a shared Task Engine, not to its own notification channel. The Task Engine owns: task type, linked record/context, client, due date, assignee (via a configurable routing rule table), status lifecycle (Open → In Progress → Completed/Escalated), and auto-escalation when a task passes its due date. All other modules (dashboard, per-user queues, client workspace) read from this same engine rather than maintaining their own task state.

Consequences: - This is a dependency every other module has on the Task Engine being built early — it should be in Phase 1, not a later add-on. - The routing rule table (task type → default assignee/role) is a real design artifact that needs to be built deliberately, ideally in collaboration with the proprietor, so that routine tasks land on junior/senior staff rather than

defaulting to the proprietor and defeating the purpose. - Escalation logic directly addresses the discovery finding about inconsistent follow-through — this is a deliberate enforcement mechanism, not just a convenience feature.

Alternatives considered: - *Per-agent notification systems* — rejected; this is what the customer explicitly asked to move away from, and it would fragment task visibility across modules instead of giving the proprietor one place to see practice-wide load.

ADR 0006 — App Form Factor: Desktop-First PWA

Status: Accepted

Context: Staff primarily work at a desktop with a monitor, not on mobile. PWAs are often built mobile-first by default, which would be the wrong assumption here.

Decision: Build as a PWA accessible both through a regular browser tab and as an installed application, with a desktop-first responsive design. Wide, multi-column layouts (dashboard, task queue, client workspace) are the primary target; mobile is a secondary, fully-functional but reflowed layout, not the primary design constraint.

Consequences: - Frontend component and layout decisions (in the eventual frontend/design work) should be evaluated at desktop breakpoints first, with mobile treated as a reflow rather than the base case. - PWA capabilities (offline task queue, push notifications) remain available for any staff member who does install it or checks tasks on mobile, without constraining the primary desktop experience.

Alternatives considered: - *Mobile-first PWA design* — rejected; would fit the COCA reference project's field-worker context, but not this practice's desk-based staff.

ADR 0007 — Technology Stack

Status: Proposed (revised)

Context: No tech stack had been decided yet. The reference project (COCA) used NestJS + PostgreSQL/PostGIS + Redis/BullMQ + AWS ECS Fargate + Turborepo — but COCA had three separate user-facing surfaces (offline-first mobile field app, ops dashboard, client portal) and a VC-backed team with dedicated DevOps. This project has a single user-facing surface (one desktop-first PWA, per ADR 0006, no client portal per §6 of the PRD) and a cost-sensitive, payback-justified budget (no dedicated technology line item per discovery). **Revision note:** the initial draft weighted this decision toward the webhook/integration workload and assumed OCR/matching would be a thin wrapper around external APIs. On reconsideration, the document-extraction and ledger/reconciliation-matching engine (BK-02, RC-02) is the actual differentiator the solution's success depends on — not the webhook plumbing — and that changes the calculus materially.

Decision:

Layer	Choice	Rationale
Backend framework	Python (FastAPI) — revised from NestJS	The extraction/matching engine is the differentiator, not a peripheral concern. Python's ecosystem (rapidfuzz, pandas for reconciliation logic, a real path to embedding-based matching later, and mature self-hosted OCR options if cloud OCR raises data-residency concerns per ADR 0004) is meaningfully stronger here than Node's equivalents. FastAPI's async support handles the webhook/I/O workload adequately even though it isn't Python's traditional strength. One language throughout avoids the operational overhead of a Node+Python hybrid for a small team.
Database	PostgreSQL	Unchanged — ACID guarantees for financial/task data
ORM	SQLAlchemy (async) + Alembic for migrations	Python-native equivalent of the Prisma/NestJS pairing
Queue	Redis + Celery (or RQ for lighter weight)	Required by ADR 0003 for async webhook processing; Python-native equivalent of BullMQ
Frontend	React + Vite + TypeScript	Unchanged — desktop-first PWA per ADR 0006; frontend language choice is independent of backend language
UI	shadcn/ui + Tailwind CSS	Unchanged
PWA layer	Workbox (service worker) + Web Push API	Unchanged
OCR	Evaluate self-hosted (Tesseract/PaddleOCR) vs. cloud API (Claude/GPT-	Ties directly to the open OCR/LLM vendor item

Layer	Choice	Rationale
Hosting	4V/Document AI) as part of the still-open OCR vendor decision — Python backend keeps both options equally viable, which a Node backend would not AWS ap-south-1 (Mumbai) — revised from Railway	Revision: Railway was the original recommendation for its lighter operational footprint vs. COCA's full AWS Fargate + Terraform setup, but Railway has no India region (closest is Singapore). Once DPDP-driven self-hosted OCR became the decision (ADR 0008), hosting also needs to be India-resident for that principle to actually hold — self-hosted OCR running outside India doesn't meaningfully improve on a cloud API with India-region processing. AWS Mumbai keeps the practice's data in-country, matches the region COCA also used for the same reason, and still allows a lightweight setup (a single EC2 instance or small ECS setup) without Fargate/Terraform-level complexity — the "lighter than COCA" principle still holds, just not via Railway specifically.
Repo structure	Single repo, two packages (api, web)	Unchanged — frontend (TS) and backend (Python) naturally separate packages regardless
CI/CD	GitHub Actions	Unchanged

Consequences: - Frontend (TypeScript/React) and backend (Python) are now different languages — loses the "one language across the stack" simplicity the original NestJS recommendation offered, in exchange for materially better tooling on the part of the system that determines whether the product actually works well. - Type-sharing between frontend and

backend requires either a generated OpenAPI client or manual type definitions — FastAPI's automatic OpenAPI generation makes this workable, but it's an explicit trade-off to note, not a wash. - Keeps self-hosted OCR genuinely viable as an option, which matters if the OCR vendor decision leans toward data-residency concerns over convenience. - Still Proposed, not Accepted — same caveat as before: needs confirmation from whoever actually builds this, since team familiarity with Python vs. Node matters as much as this analysis.

Alternatives considered: - *NestJS (original recommendation)* — superseded by this revision; was reasonable when the webhook/integration workload was weighted as primary, less so once the matching engine is recognized as the differentiator. - *Hybrid: NestJS app layer + Python matching microservice* — genuinely viable, best-of-both technically, but adds a second runtime/deployment unit that's real overhead for a small team. Worth reconsidering if the team ends up being large enough to support polyglot services comfortably. - *Full COCA-equivalent stack (AWS ECS Fargate + Terraform + Turborepo)* — still rejected as disproportionate to this project's scale and budget.

Status note: This ADR changed direction once between drafts based on which workload was weighted as primary — a useful reminder to lock down “what actually determines success” before finalizing stack decisions, not after.

ADR 0008 — Self-Hosted OCR Engine: PaddleOCR (PP-StructureV3)

Status: Accepted

Context: ADR 0004 established the platform as a thin layer over Tally/Dropbox to narrow the DPDP Act surface, but left the OCR/LLM vendor decision open. Given DPDP Act stringency, the decision was made to self-host OCR rather than send client financial documents to a third-party cloud OCR/LLM API. Documents are confirmed to be ~99% printed PDFs (mostly clean invoices) with ~1% handwritten (per discovery), and extraction needs to cover structured fields (GSTIN, vendor, amounts, dates) and tabular line items, not just raw text.

Decision: Use **PaddleOCR with the PP-StructureV3 module** as the primary, self-hosted OCR/extraction engine. PP-Structure provides built-in layout analysis, table recognition, and key-value extraction — directly relevant to invoice line-item extraction, which is the hardest part of this pipeline and not something raw OCR text output handles well. Run CPU-only initially, given document volume is moderate and processing is already asynchronous (ADR 0003) with no real-time latency requirement — GPU is a scale-up option, not a day-one cost.

Two complementary pieces, not primary engines: - **EasyOCR** as a secondary path specifically for the ~1% handwritten documents, where PaddleOCR is weaker. - **invoice2data** (template-based) as a fast-path optimization for previously-seen vendor formats over time — ties into the existing confidence-scoring design (BK-05): a known vendor template gives high confidence and can auto-stage, while an unrecognized layout goes through the full PaddleOCR pipeline and a lower confidence score.

Consequences: - No client financial document is ever sent to a third-party OCR/LLM API — directly satisfies the DPDP data-residency principle established in ADR 0004, provided hosting

itself is also India-resident (see ADR 0007 revision). - Starting CPU-only keeps infrastructure cost low; if throughput becomes a genuine bottleneck as volume scales toward 500 clients, GPU provisioning is a targeted upgrade, not a rebuild. - PaddleOCR's accuracy should be validated against a representative sample (20-50 real invoices across different vendor formats) before committing further — this is standard practice for evaluating open-source OCR fit, not just a formality, since production accuracy on this specific practice's actual documents is the only number that matters. - DocTR was considered as an alternative but confirmed to lack table structure recognition in its current release — a real gap for line-item extraction specifically, so it wasn't chosen as primary. - Open-weight VLMs (Qwen2.5-VL, DeepSeek-OCR, etc.) remain a future option for hard/low-confidence cases, but are not the primary engine — numeric accuracy on financial figures needs validation before trusting VLM-style extraction over deterministic OCR, and GPU cost/latency are real constraints.

Alternatives considered: - *Cloud OCR/LLM API (Claude, GPT-4V, AWS Textract, Azure Document Intelligence, Google Document AI)* — rejected as the primary path given DPDP stringency, even though some of these offer India-region processing (e.g., AWS Textract via Mumbai). Self-hosting removes the question entirely rather than relying on a vendor's regional processing guarantee. - *Tesseract as primary* — rejected; CPU-only and fast, but raw-text-only output would require substantial custom logic to reconstruct table/line-item structure that PP-Structure provides out of the box. - *DocTR as primary* — rejected due to confirmed lack of table structure recognition in the current release.

ADR 0009 — Deployment Model: Single-Tenant, Per-Customer Instances

Status: Accepted

Context: The platform is being built for the first customer (Venture Assist / Srivatsan & Associates) but is planned to roll out to other CA practices once this deployment succeeds. The initial instinct was to design for multi-tenancy (shared infrastructure, tenant isolation via a practice-level entity above CLIENT, row-level security). The customer clarified this is explicitly **not** the model: each future practice gets a fully separate deployment, not a shared multi-tenant platform.

Decision: Build and deploy as single-tenant, one full stack (app, database, connectors) per practice. No tenant/practice entity is added above CLIENT in the data model — the ER diagram (\$ audit-platform-ER-*.mermaid) stays as originally designed. Each practice's Tally connection, GSP credentials, service catalog, and branding are deployment-specific configuration, not shared or tenant-scoped data.

Consequences: - **No data-model change needed** — the existing ER diagrams and PRD module list are correct as-is; the earlier suggestion to add a PRACTICE tenant layer is retracted. - **Data isolation is structural, not app-enforced** — separate databases per deployment means no risk of cross-practice data leakage from an application bug, which is a stronger position than row-level security under DPDP scrutiny. - **The real engineering shift is toward repeatable deployment, not tenant isolation.** Infrastructure-as-code (e.g. Terraform) should be adopted from this first deployment, even though there's only one instance today — hand-configuring AWS Mumbai once is fine; hand-configuring it identically for a second, third,

and fourth customer is not. - **The Dev Readiness Checklist (still pending, per the Documentation Roadmap) should be written as a reusable “new practice deployment” runbook**, not a one-time setup document — since it will literally be re-run for each new customer. - **An update/patch strategy across multiple live deployments needs deciding**, even at small scale (2-3 customers). Options range from manual redeploy per instance to a proper CI/CD release pipeline pushing to all instances. Worth deciding this now, while only one deployment exists, rather than retrofitting a release process once several are live and drifting apart. - **Per-deployment configuration must be cleanly separated from code** — Tally connection details, GSP credentials, service catalog, and any branding differences belong in environment/config, not hardcoded, so the same codebase deploys to a new environment without code changes. - **Branding/white-labeling is naturally simple** under this model, since nothing is shared between deployments — no additional design work needed to support per-practice branding if that’s ever wanted. - Infrastructure cost scales linearly per customer (each deployment needs its own hosting and OCR compute) rather than being amortized across tenants — worth factoring into how future customer pricing is structured, though that’s a business decision outside this ADR’s scope.

Alternatives considered: - *Multi-tenant shared platform* — this was the initial direction explored based on an assumption about the rollout plan; explicitly rejected once the customer clarified the actual model. Retained here as a considered-and-rejected alternative since the reasoning (tenant entity, RLS, shared GSP) is real work that would have been wasted if built.

ADR 0010 — Base Product with Per-Practice Customization and Upstream Promotion

Status: Accepted

Context: ADR 0009 established single-tenant, per-practice deployments and flagged the update/patch strategy across multiple live instances as an open item. The customer clarified the model directly: build one “base product” and deploy customized versions per practice, with generalizable customizations pushed back into the base so future (and ideally existing) deployments benefit.

Decision: Maintain a single base product codebase as the source of truth for all generic, reusable functionality — every agent, the task engine, and the PWA screens defined in the PRD. Each practice’s deployment starts from the base and layers on customization only where genuinely necessary. When a customization proves broadly useful rather than specific to one practice’s idiosyncrasy, it gets generalized — typically as a configuration option, not hardcoded logic — and merged back into the base, so future deployments inherit it and existing deployments can adopt it.

Practical structure: - **Configuration is the default and cheapest form of customization.** Per ADR 0009, Tally connection, GSP credentials, service catalog, and branding already live in config, not code — this is customization that was never divergent in the first place, so it never needs “pushing back.” - **Where code-level customization is genuinely needed, isolate it behind clear extension points** (e.g. a defined hook/plugin pattern, or a

practices/{practice-name}/ override directory) rather than editing core modules directly. This keeps the base/customization boundary legible and makes eventual promotion to base a clean extraction rather than an untangling exercise. - **A lightweight periodic review of practice-specific customizations for generalization candidates** should be part of the ongoing process — this is what actually makes the “push to base” half of the model happen, rather than customizations quietly accumulating as one-off technical debt.

Consequences: - Resolves the update/patch strategy open item from ADR 0009 — updates to the base flow to existing practice instances the same way any software update would, since deployments aren’t meant to diverge arbitrarily from it. - Requires real discipline to keep the boundary clean: if practice-specific edits get made directly against core modules without going through an extension point, future base updates become risky to apply (merge conflicts, silent breakage) — this is the main way the model fails in practice. - The extension-point pattern needs concrete definition — actual code structure, not an abstract principle — before the second deployment happens. Worth doing this during or immediately after the first deployment build, not designing it in the abstract now with no real customization yet to learn from. - The Dev Readiness Checklist (still pending) should include defining this extension-point structure as an explicit task, since it’s foundational to every deployment after the first.

Alternatives considered: - *Per-practice long-lived forks with manual cherry-picking* — simplest at 1-2 practices, but merge/rebase overhead compounds with each additional deployment and risks silent drift between instances. Rejected as the primary strategy, though it may resemble what happens informally at very small scale before proper tooling is in place. - *Fully generic base with no practice-specific customization allowed* — rejected as unrealistic; some practices will have genuine one-off needs (a specific report format, an unusual workflow quirk) that don’t belong in the base product for every customer. The extension-point pattern accommodates this without contaminating the base.

Open — Not Yet Decided

- **DPDP Act specific obligations** for the platform’s own metadata layer — needs legal/compliance input.
- **PaddleOCR accuracy validation** against a representative sample of real practice documents (20-50 invoices across vendor formats) — needed before ADR 0008 can be considered fully proven in practice, not just sound in theory.
- **Infrastructure-as-code tooling choice** (Terraform vs. alternatives) for repeatable per-practice deployment, per ADR 0009.

ER Diagram — Practice Automation Platform

Prepared for Venture Assist / Srivatsan & Associates | Draft v0.1

Changelog

Version	Date	Summary
v0.1	2026-08-16	Initial entity model, split into two diagrams (core platform, compliance & billing) per the PRD's 14 modules. Sources: audit-platform-ER-core.mermaid, audit-platform-ER-compliance-billing.mermaid.

Why two diagrams, not one

23 entities is more than fits legibly in a single diagram. Split by the same logical boundary the PRD already uses: the **core platform** (identity, tasks, documents, bookkeeping) that most agents write to, and **compliance & billing** (reconciliation, validation, TDS, bank, working papers, notices, communication, billing) that mostly reads from the core and writes its own domain-specific records. CLIENT is the shared anchor entity across both.

This mirrors the PRD's own module grouping (\$5) and keeps each diagram traceable back to a specific set of requirement IDs.

Diagram 1 — Core Platform

Source: audit-platform-ER-core.mermaid

Covers: Identity & Access (ID), Task Engine (TE), Tally Connector (TC), Email Intake (EI), Document Intake (DI), Bookkeeping/Voucher Entry (BK).

Entity	Purpose	Key PRD traceability
CLIENT	Master client record — anchor entity for the whole system	ID, CB
USER	Staff member (Proprietor / Senior / Junior)	ID-01
TASK	Every human-touch item any agent raises, per ADR 0005	TE-01 to TE-09
DOCUMENT	A file received via email/Dropbox, before	EI, DI-01 to DI-04

Entity	Purpose	Key PRD traceability
EXTRACTED_DATA	extraction OCR/parsing output from a document (PaddleOCR per ADR 0008)	DI-02, DI-03
TALLY_COMPANY	A client's company within the practice's Tally Cloud, or the secondary backup-import path	TC-01 to TC-08
VOUCHER	A draft or posted Tally voucher, with confidence score	BK-01 to BK-08
VENDOR_MAPPING	Learned vendor-name-to-ledger matches, used by the fuzzy-matching engine	BK-02
EMAIL_SENDER_MAP	Learned sender-address-to-client mapping	EI-02, EI-06

Notable relationships:

- DOCUMENT → EXTRACTED_DATA is 1:1 — each document produces one extraction result (which may be low-confidence and routed to a task).
- EXTRACTED_DATA → VOUCHER is 1:many — a single document (e.g. a multi-line invoice) can generate multiple draft vouchers.
- TASK.assigned_id references USER, but TASK itself is not tied to a single entity type — it links to whichever record raised it (document, voucher, reconciliation item, etc.) via a generic reference, kept out of this diagram for clarity and left as an implementation-level polymorphic reference rather than a rigid foreign key, since the whole point of the Task Engine (ADR 0005) is that any module can raise one.

Diagram 2 — Compliance & Billing

Source: audit-platform-ER-compliance-billing.mermaid

Covers: Reconciliation (RC), Validation/Compliance (VC), TDS/TCS (TDS), Bank Reconciliation (BR), Working Paper (WP), Notice/Scrutiny Tracker (NT, future), Client Communication (CC), Client Profiling & Billing (CB).

Entity	Purpose	Key PRD traceability
CLIENT	Shared anchor with Diagram 1	—
RECONCILIATION	A GSTR-2A/2B match result for a client/period	RC-01 to RC-05
VALIDATION_CHECK	A pre-filing check result	VC-01 to VC-06

Entity	Purpose	Key PRD traceability
	(GSTR-1 vs 3B, HSN, TDS)	
TDS_RECORD	A TDS deduction/deposit match for a client/period	TDS-01 to TDS-04
BANK_ACCOUNT / BANK_TXN	A client's bank account and its transactions	BR-01 to BR-04
WORKING_PAPER	An engagement's working paper, draft or finalized	WP-01 to WP-04
NOTICE	A GST/IT notice — future scope, deprioritized per discovery	NT-01 to NT-04
COMMUNICATION	A logged client communication (reminder, query)	CC-01 to CC-05
SERVICE_CATALOG	The practice's service/price-tier definitions	CB-02
SERVICE_OVERRIDE	A client-specific price override against the catalog	CB-03
INVOICE	A billing-cycle invoice, GST-compliant	CB-05, CB-09
BILLABLE_EVENT	A single billable line item, generated from a task/agent event	CB-04
GSP_CREDENTIAL	Per-client GSTIN authorization with the chosen GSP (WhiteBooks or similar)	RC-01, ADR 0002

Notable relationships:

- INVOICE → BILLABLE_EVENT is 1:many, and each BILLABLE_EVENT optionally links back to the TASK that generated it (via task_id) — this is what makes CB-04 (billing-trigger mapping) traceable, rather than a black box.
- SERVICE_OVERRIDE sits between CLIENT and SERVICE_CATALOG as a many-to-one bridge — a client can have multiple overrides (one per service type), each referencing the standard catalog item it overrides.
- GSP_CREDENTIAL is 1:1 with CLIENT in this diagram, reflecting that GSTN's OTP/consent step (ADR 0002) is per-GSTIN, not shared across clients.

What's deliberately not modeled yet

- Full column-level schema (types, constraints, indexes) — this is an entity-relationship model for architecture review, not a migration script. That level of detail belongs in the actual Prisma/SQLAlchemy schema once development starts.

- The generic TASK – source-record link is left as a conceptual polymorphic reference rather than drawn out, since forcing it into a rigid ER relationship would either require a separate join table per source-entity type (cluttering both diagrams) or misrepresent the Task Engine’s actual flexibility.
- Notice is included for schema completeness even though it’s explicitly future/deprioritized scope, so the model doesn’t need revisiting if the practice’s notice volume changes later.

API Specification — Practice Automation Platform

v0.1

Changelog

Version	Date	Summary
v0.1	2026-08-16	Initial spec: ~58 endpoints across 12 domains, traced to PRD requirement IDs and the ER diagram entities. Auth/error/pagination conventions defined once, not repeated per endpoint.

1. How this spec is meant to be used

This is a design-time reference, not the final machine-readable artifact. Per CG9 (Coding Guidelines): every request/response shape is a Pydantic model in the actual codebase, and FastAPI auto-generates the real OpenAPI 3.0 JSON from those models at runtime — this differs from COCA’s spec-first approach (hand-written OpenAPI YAML as the source of truth), because FastAPI’s idiomatic pattern is code-first. This document exists so the API surface can be reviewed and the Sprint Plan sequenced against it *before* that code exists, not as a permanent parallel spec to keep manually in sync afterward. Once built, `docs/api/openapi.json` (FastAPI-generated) becomes the actual source of truth.

2. Conventions (defined once)

Base path: `/api/v1`

Auth: JWT bearer token, obtained via `POST /auth/login`. Per the Security Standard §1, tokens expire after a fixed idle period (8 hours default). Every endpoint below other than `/auth/login` requires a valid token; role requirements are noted per endpoint per the routing/authorization model in Security Standard §2.

Roles: `proprietor | senior | junior` (per ID-01). “Any staff” means all three.

Standard error shape:

```
{ "error_code": "voucher_not_found", "message": "Voucher does not exist or you don't have access.", "detail": null }
```

Pagination (list endpoints): `?page=1&page_size=25`, response wraps results as `{ "items": [...], "total": 142, "page": 1, "page_size": 25 }`.

Money fields: always `Decimal`-serialized as strings (e.g. `"18400.00"`), never JSON floats — per CG5.

3. Auth & Identity (ID)

Method	Path	Role	Request	Response	Traces to
POST	/auth/login	—	{email, password}	{access_token, expires_at, user}	ID-01
POST	/auth/logout	Any staff	—	204	ID-04
GET	/users/me	Any staff	—	User	ID-01
GET	/users	Proprietor	—	User[]	ID-01
GET	/audit-log	Proprietor	?actor_id=&event_type=&from=&to=	AuditEvent[] (paginated)	ID-05, Security \$5

4. Task Engine (TE)

Method	Path	Role	Request	Response	Traces to
GET	/tasks	Any staff	?assignee_id=&client_id=&status=&type=	Task[] (paginated)	TE-07
GET	/tasks/{id}	Any staff	—	Task	TE-01
PATCH	/tasks/{id}	Any staff (own) / Senior+ (reassign)	{status?, assignee_id?, outcome?}	Task	TE-03, TE-06
POST	/tasks/{id}/complete	Assignee	{outcome}	Task	TE-06
GET	/dashboard/summary	Any staff	—	DashboardSummary (KPIs, staff workload)	PM-01, PM-02
GET	/admin/routing-rules	Proprietor	—	RoutingRule[]	TE-02
PUT	/admin/routing-rules/	Proprietor	{default_role, escalate_a	RoutingRule	TE-02

Method	Path	Role	Request	Response	Traces to
	{task_type }		fter_hours }		

Task includes: id, type, client_id, linked_record_type, linked_record_id, assignee_id, due_at, status, outcome, created_by_agent.

5. Tally Connections (TC — mostly internal, admin-facing subset)

Method	Path	Role	Request	Response	Traces to
GET	/admin/ tally- connections	Proprietor	—	TallyConnections[]	TC-05, TC-08
POST	/admin/ tally- connections/ {client_id }/sync	Proprietor	—	202 (async)	TC-01–TC-03
POST	/admin/ tally- connections/ {client_id }/restore- backup	Proprietor	multipart file	202 (async)	TC-06, TC-07

6. Email Intake (EI)

Method	Path	Role	Request	Response	Traces to
GET	/admin/ sender- mappings	Senior+	—	SenderMapping[]	EI-02, EI-06
POST	/admin/ sender- mappings	Senior+	{sender_domain, client_id}	SenderMapping	EI-06
PATCH	/admin/ sender- mappings/ {id}	Senior+	{client_id }	SenderMapping	EI-06

7. Document Intake (DI)

Method	Path	Role	Request	Response	Traces to
POST	/webhooks/dropbox	— (Dropbox signature verified, not user auth)	Dropbox notification payload	200 immediately, per the 10s constraint	DI-01, DI-01a
GET	/clients/{id}/documents	Any staff	?period=&doc_type=	Document []	DI-02, DI-03
GET	/documents/{id}	Any staff	—	Document (incl. extra cted_data)	DI-02
GET	/clients/{id}/documents/checklist	Any staff	?period=	DocumentChecklist	DI-04

8. Bookkeeping (BK)

Method	Path	Role	Request	Response	Traces to
GET	/vouchers	Any staff	?client_id=&status=confidence_level=	Voucher [] (paginated)	BK-05
GET	/vouchers/{id}	Any staff	—	Voucher (incl. source document ref)	BK-04
POST	/vouchers/{id}/approve	Any staff	—	Voucher — posts to Tally (runs CG7 dup-check)	BK-06, BK-07, BK-08
POST	/vouchers/{id}/reject	Any staff	{reason}	Voucher	BK-06
POST	/vouchers/{id}/create-ledger	Senior+	{ledger_name}	Voucher	BK-03

9. Client Communication (CC)

Method	Path	Role	Request	Response	Traces to
GET	/communications	Any staff	?client_id=&status=	Communication[]	CC-04
GET	/communications/pending-approval	Proprietor	—	Communication[]	CC-03
POST	/communications/{id}/approve	Proprietor	{edited_body?}	Communication — sends	CC-03

10. Reconciliation (RC)

Method	Path	Role	Request	Response	Traces to
POST	/clients/{id}/reconciliation/run	Senior+	{period}	202 (async — calls GSP per ADR 0002)	RC-01, RC-02
GET	/reconciliation/mismatches	Any staff	?client_id=&reason=	MismatchItem[] (paginated)	RC-04
GET	/reconciliation/mismatches/{id}	Any staff	—	MismatchItem	RC-04
POST	/reconciliation/mismatches/{id}/resolve	Senior+	{resolution_note}	MismatchItem	RC-04
GET	/clients/{id}/reconciliation/summary	Any staff	?period=	ReconciliationSummary (auto-confirmed ITC total)	RC-03, RC-05

11. Validation / Compliance (VC)

Method	Path	Role	Request	Response	Traces to
POST	/clients/{id}/validation/run	Senior+	{period}	202 (async)	VC-01-VC-04
GET	/validation/checks	Any staff	?client_id=&result=	Validation Check[]	VC-05
GET	/clients/{id}/filing-readiness	Any staff	?period=	{status: "cleared" \ "blocked", blocking_checks: []}	VC-06

12. TDS/TCS (TDS)

Method	Path	Role	Request	Response	Traces to
POST	/clients/{id}/tds/run	Senior+	{quarter}	202 (async)	TDS-01, TDS-02
GET	/tds/records	Any staff	?client_id=&status=	TdsRecord[]	TDS-03
POST	/tds/records/{id}/resolve	Senior+	{resolution_note}	TdsRecord	TDS-03
GET	/clients/{id}/tds/draft-return	Any staff	?quarter=	TdsDraftReturn	TDS-04

13. Bank Reconciliation (BR)

Method	Path	Role	Request	Response	Traces to
GET	/clients/{id}/bank-accounts	Any staff	—	BankAccount[]	BR-01
POST	/clients/{id}/bank-accounts	Senior+	{bank_name, account_ref}	BankAccount	BR-01
GET	/bank-	Any staff	?	BankTransa	BR-02

Method	Path	Role	Request	Response	Traces to
	transactions		account_id=&status=unmatched	transaction[]	
POST	/bank-transaction/{id}/resolve	Any staff	{resolution_note}	BankTransaction	BR-03
GET	/bank-accounts/{id}/statement	Any staff	?period=	BankReconciliationStatement	BR-04

14. Working Paper (WP)

Method	Path	Role	Request	Response	Traces to
GET	/working-papers	Any staff	?client_id=&status=	WorkingPaper[]	WP-01
GET	/working-papers/{id}	Any staff	—	WorkingPaper	WP-02
PATCH	/working-papers/{id}	Proprietor	{judgment_notes}	WorkingPaper	WP-03
POST	/working-papers/{id}/finalize	Proprietor	—	WorkingPaper (status → finalized, filed to Dropbox)	WP-04

15. Practice Management (PM)

Method	Path	Role	Request	Response	Traces to
GET	/staff/workload	Any staff	—	StaffWorkload[]	PM-02
GET	/capacity	Any staff	—	{available_staff: int, at_risk_clients: []}	PM-03
GET	/deadlines/calendar	Any staff	?from=&to=&client_id=	Deadline[]	PM-04

16. Client Profiling & Billing (CB)

Method	Path	Role	Request	Response	Traces to
GET	/clients	Any staff	? status=&search=	Client[] (paginated)	CB-01
POST	/clients	Senior+	ClientCreateRequest	Client	CB-01
GET	/clients/{id}	Any staff	—	Client (incl. services subscribed)	CB-01
PATCH	/clients/{id}	Senior+	ClientUpdateRequest	Client	CB-01
POST	/clients/{id}/deactivate	Proprietor	—	Client	CB-01
GET	/service-categories	Any staff	—	ServiceCategory[]	CB-02
POST	/service-categories	Proprietor	{name}	ServiceCategory	CB-02
GET	/service-catalog	Any staff	? category_id=	ServiceCatalogItem[]	CB-02
POST	/service-catalog	Proprietor	ServiceCatalogItemCreateRequest	ServiceCatalogItem	CB-02
PATCH	/service-catalog/{id}	Proprietor	{price?, active?}	ServiceCatalogItem	CB-02, CB-08
GET	/clients/{id}/service-overrides	Any staff	—	ServiceOverride[]	CB-03
POST	/clients/{id}/service-overrides	Proprietor	{catalog_item_id, override_price, reason}	ServiceOverride	CB-03
GET	/invoices	Any staff	? client_id=&status=	Invoice[] (paginated)	CB-07
GET	/invoices/{id}	Any staff	—	Invoice (incl. billable events, GST breakup)	CB-05, CB-09

Method	Path	Role	Request	Response	Traces to
POST	/invoices/{id}/approve	Proprietor	—	Invoice — sends	CB-06
GET	/invoices/draft	Proprietor	?period=	Invoice[] (current cycle drafts)	CB-05

17. What's deliberately not in this spec

- Endpoints for Notice/Scrutiny Tracker — deferred scope, per the Feature Backlog's "Future" phase.
- Webhook/callback endpoints for the GSP itself (if WhiteBooks or the chosen provider supports async callbacks rather than polling) — not yet designed, since the GSP vendor selection (RC-01d) hasn't run its sandbox spike yet.
- Bulk import/export endpoints — not requested in the PRD; add only if a real need surfaces during Phase 0/1.

Wireframes — Practice Automation Platform

Prepared for Venture Assist / Srivatsan & Associates | v0.2 — Design B (Capacity) applied

Changelog

Version	Date	Summary
v0.1	2026-08-16	First pass: 4 core screens wireframed (Dashboard, My Tasks, Client Workspace, Bookkeeping Review). Deliberately low-fidelity — structural layout only, no visual design applied yet.
v0.1 (rev)	2026-08-16	Added Admin screen: client master list with add/edit/deactivate, add-client form, service catalog (grouped by category) with add/edit/deactivate, add-service and add-category forms. Now 5 of 9 screens covered.
v0.1 (rev 2)	2026-08-16	Expanded service catalog from 2 to 7 categories, drawn from the actual service lines listed on the practice's own websites (Venture Assist, Srivatsan & Associates) — not all of which map to a current automation agent.
v0.1 (rev 3)	2026-08-16	All 9 screens complete: added Reconciliation Review, Client Communication (approval queue + log), Reports & Working Papers, Billing (draft approval + history), and Admin's remaining sub-screens (Assignment Rules, Connections). First full pass across the whole PWA scope.
v0.2	2026-08-16	Customer selected Design B (Capacity) from the three visual directions presented. Applied across all 9 screens: teal/amber palette, Inter

Version	Date	Summary
		typography, rounded cards, and the radial capacity gauge as the Dashboard's signature element. Semantic badge coloring corrected in a few places where the low-fi version's neutral styling didn't map cleanly onto Design B's green/coral system (reconciliation mismatch reasons, selected-row highlights).

Visual design status

Design B (Capacity) — teal #0F3D3E + amber #E8A33D, Inter typography, rounded 14px cards, radial capacity gauge as the Dashboard's signature element — is now applied consistently across all 9 screens, per the customer's selection from the three directions presented earlier.

File: `audit-platform-wireframes-v0.1.html` — open directly in a browser; click sidebar items to switch between all 9 screens.

Screen coverage

#	Screen	Status	Covers (PRD traceability)
1	Dashboard	Wireframed	PM-01 to PM-06
2	My Tasks	Wireframed	TE-07, TE-01 to TE-09
3	Client Workspace	Wireframed	Cross-module client view — DI, BK, RC, CB status
4	Bookkeeping Review	Wireframed	BK-05, BK-06
5	Reconciliation Review	Wireframed	RC-03, RC-04 (auto-confirm + mismatch queue)
6	Client Communication	Wireframed	CC-02 to CC-05 (approval queue + log)
7	Reports & Working Papers	Wireframed	WP-01 to WP-04
8	Billing	Wireframed	CB-05 to CB-09 (draft approval + invoice)

#	Screen	Status	Covers (PRD traceability)
9	Admin — Clients	Wireframed	history) CB-01 (client master profile, add/edit/deactivate)
9b	Admin — Service catalog	Wireframed	CB-02 (service catalog, tiers, add/edit/deactivate, categories)
9c	Admin — Assignment rules	Wireframed	TE-02 (task routing rule table)
9d	Admin — Connections	Wireframed	TC-05, TC-08 (Tally + GSP connection health per client)

All 9 planned PWA screens are now wireframed. Notice/Scrutiny Tracker remains deliberately excluded — deprioritized per discovery (negligible notice volume) and not part of the current build scope.

Design decisions embedded in these wireframes

- **Sidebar navigation, not tabs or a hamburger menu** — matches the desktop-first principle from ADR 0006; staff working at a monitor benefit from persistent navigation rather than a collapsed mobile pattern.
- **Dashboard is the default landing screen** — reflects PM-01's cross-client, cross-staff visibility as the proprietor's primary view, not a generic home page.
- **Bookkeeping Review shows the source document alongside the staged voucher list** — this was a deliberate layout choice, not incidental: BK-06 requires staff to visually verify the extraction against the original document before approving, so the two need to sit side by side rather than in separate screens.
- **Client Workspace uses a status-row pattern** (documents / bookkeeping / reconciliation / filing) rather than a single combined status — each module can be in a different state simultaneously, and hiding that behind one summary status would obscure exactly the kind of detail a senior staff member needs at a glance.
- **Task cards on My Tasks show the agent that raised them** (Reconciliation Agent, Bookkeeping Agent, etc.) — reinforces the ADR 0005 principle that every task traces back to a specific source, not a generic notification.
- **Admin uses an inline expandable form rather than a separate page** for adding/editing clients and services — keeps the list visible for reference while filling the form, and avoids a page-navigation round trip for a frequent, simple action.
- **Deactivate, not delete** — client and service records use an active/inactive status toggle rather than deletion, preserving historical data (past invoices, task history,

reconciliation records) for a client or service that's no longer active. This matters for audit trail integrity, not just convenience.

- **Service catalog is grouped by category** (GST compliance, Income tax, etc.) rather than one flat list — reflects that CB-02's catalog is being built from scratch, and categories give the proprietor a way to organize pricing decisions by compliance area rather than an undifferentiated list.
 - **Service categories now reflect the practice's actual marketed service lines**, cross-checked against both of the practice's own websites, not just the automation-agent scope discussed earlier. This surfaced real gaps: Entity Setup & Registration, Corporate Finance & Advisory, and Services for Non-Residents weren't previously represented anywhere in the PRD. One nuance worth flagging directly to the practice: both sites market "Income Tax & GST Representations" (notice/scrutiny response) as an active service line, even though the discovery call described actual notice volume as negligible — the service catalog now includes it as a billable line item regardless, since billing needs to cover what's marketed and occasionally sold, not only what the current automation agents handle.
 - **Reconciliation Review only shows what needs a human** — matched line items never appear in this queue at all (RC-03 auto-confirms them); the screen surfaces a one-line summary of what got auto-confirmed instead, so staff aren't left wondering whether reconciliation ran at all.
 - **Client Communication splits "awaiting approval" from "log"** rather than one combined feed — the approval queue is an action list (things staff must do), the log is a reference view (things already sent). Conflating them would bury actionable items in a longer historical list.
 - **Assignment rules show the escalation path inline**, not just the default assignee — this makes the ADR 0005 enforcement mechanism visible and editable, rather than a hidden background rule nobody can inspect or adjust as the practice's needs change.
 - **Connections screen distinguishes primary vs. secondary Tally path per client** (per ADR's TC-06/TC-08) and separately tracks GSP authorization status — these are two different failure modes (a stale backup import vs. an expired GSTN consent) that need different remediation, so collapsing them into one generic "connected/disconnected" status would hide which problem is actually happening.
-

Next steps

- All 9 screens are wireframed — this closes the layout/structure pass across the full PWA scope.
- Three visual design directions are now available for the customer to pick from (see `audit-platform-design-directions-v0.1.md`) — Ledger, Capacity, and Register, each applied to the Dashboard for direct comparison.
- Once a direction is chosen (or a hybrid), apply it consistently across all 9 wireframed screens before any staff-facing prototype testing.

Visual Design Directions — Practice Automation Platform

Three options for the customer to review, same Dashboard content in each

All three render the exact same layout and data from the low-fidelity wireframes — only the visual language changes. Open each HTML file directly in a browser to compare.

A — Ledger

`audit-platform-design-A-ledger.html`

Grounded in: traditional accounting precision — ruled ledger paper, ink and brass fittings of an old accounting office.

- **Color:** ink navy (#1B2A4A) + brass/gold accent (#B08D57), warm paper white background, muted sage for “matched,” soft rust for flagged items
- **Type:** Fraunces (a bookish serif) for headings only, used sparingly; IBM Plex Sans for body; IBM Plex Mono for all data values
- **Signature element:** status indicators styled as small circular “stamps” rather than generic pill badges — evokes the rubber-stamp aesthetic of physical ledger sign-off
- **Feel:** trustworthy, established, precise — leans into the firm’s accounting heritage rather than away from it

B — Capacity

`audit-platform-design-B-capacity.html`

Grounded in: the project’s own central narrative — scaling from 100 to 500 returns without adding headcount.

- **Color:** deep teal (#0F3D3E) + warm amber accent (#E8A33D), cool off-white background, coral for overdue/alert states
- **Type:** Inter throughout — fully modern, software-native, no serif
- **Signature element:** the “GST returns this month” KPI is rendered as a radial capacity gauge, not a plain number — makes the practice’s actual growth goal the visual hero of the dashboard, not just a stat buried in a row of cards
- **Feel:** modern, systems-native, growth-oriented — reads like software the practice is scaling into, not accounting-software-as-usual

C — Register

`audit-platform-design-C-register.html`

Grounded in: the vernacular of official Indian compliance paperwork — GST/IT government forms, stamped registers, dossier files.

- **Color:** deep maroon/oxblood (#6B1F2A) — evokes official stamps and seals on Indian government documents — with slate gray and mustard accents

- **Type:** IBM Plex Mono for all codes, numbers, and headings (evokes stamped serial numbers on official forms); Public Sans for prose/labels
 - **Signature element:** a colored left-edge bar per table row (forest green = normal, mustard = flagged) — like tabbed physical file folders in a registry
 - **Feel:** official, procedural, exacting — leans into the gravitas of statutory compliance work rather than softening it
-

How these differ from generic AI-generated defaults

Per current design conventions worth actively avoiding: none of these use warm cream + terracotta clay (the most common AI default), a near-black background with a single neon accent, or an undifferentiated broadsheet-serif treatment applied without reason. Each palette and typographic choice is tied to something specific about this practice's actual work — ledgers, growth targets, or government compliance forms — not a template applied regardless of subject.

Note on brand alignment

These are ground-up proposals, since the practice's actual brand colors weren't available in this session (no browser screenshot capability). If the practice has existing brand colors from Venture Assist or Srivatsan & Associates they want reflected, share them and whichever direction is chosen can be re-colored around that palette without changing its underlying structure or signature element.

Next step

Once a direction (or a hybrid) is chosen, apply it across all 9 wireframed screens — this mockup intentionally covers only the Dashboard, since that's enough to judge the direction without building the full set three times over.

Coding Guidelines — Practice Automation Platform

Backend (Python / FastAPI) | v0.1

Changelog

Version	Date	Summary
v0.1	2026-08-16	Initial set: CG1–CG11, covering linting, typing, secrets, financial precision, external-API resilience, duplicate prevention, the Task Engine contract, testing, and the base/customization boundary from ADR 0010.

Rules are numbered CG1–CG11. All are non-negotiable unless a rule explicitly says otherwise. Frontend (React/TypeScript) conventions are a separate document, not covered here.

CG1 — Linting & formatting: ruff, zero warnings before commit

ruff is the sole linter and formatter for services/api/.

```
# pyproject.toml
[tool.ruff]
line-length = 100
target-version = "py311"

[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B", "SIM"]
```

CI gate: `ruff check . --no-fix` must exit 0 before merge.

CG2 — Type hints required on every function signature

Every parameter and return type must be annotated. Use `from __future__ import annotations` at the top of every file.

✗ Bad:

```
def match_vendor(name, client_id):
    ...
```

✓ Good:

```
from __future__ import annotations

def match_vendor(name: str, client_id: str) -> VendorMatchResult |
None:
    ...
```

CG3 — Function length: 50 lines maximum

Extract helpers before hitting the limit. This applies especially to agent logic (Bookkeeping, Reconciliation) where “one function does extraction, matching, and posting” is the most common way this gets violated — those are three functions, not one.

CG4 — Secrets and configuration: centralized settings, never raw `os.environ`

All configuration — database URL, Redis URL, Tally connector credentials, GSP client ID/secret, OCR engine config — goes through a single pydantic-settings-based `Settings` object. Never call `os.environ[...]` or `os.environ.get(...)` outside `config.py`.

```
# services/api/config.py
from __future__ import annotations
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str
    redis_url: str
    gsp_client_id: str
    gsp_client_secret: str
    tally_connector_host: str
    log_level: str = "INFO"

    class Config:
        env_file = ".env"

settings = Settings() # type: ignore[call-arg]
```

Every new setting requires a corresponding placeholder in `.env`. example, committed in the same PR. No credential — GSP secret, Tally auth token, database password — is ever committed to git in any form, including in test fixtures or comments.

CG5 — Money is `Decimal`, never `float`

Every monetary amount — invoice totals, GST variance figures, TDS deduction amounts, billing line items — is a `Decimal`, sourced from strings or integers, never from a float literal or float

arithmetic. This is not a style preference: float arithmetic on currency produces silent rounding errors that surface as real reconciliation mismatches.

✗ Bad:

```
variance = book_amount - gstr_amount  # both floats
```

✓ Good:

```
from decimal import Decimal
```

```
variance: Decimal = Decimal(str(book_amount)) -  
Decimal(str(gstr_amount))
```

Pydantic models representing money use `Decimal` as the field type, not `float`.

CG6 — External API calls must have explicit timeout, retry, and failure isolation

Every call to Tally, the GSP (WhiteBooks or equivalent), or the OCR pipeline must specify an explicit timeout, use a bounded retry with backoff, and fail into a Task (per CG8) rather than raising an unhandled exception into the request path.

```
import httpx  
from tenacity import retry, stop_after_attempt, wait_exponential  
  
@retry(stop=stop_after_attempt(3), wait=wait_exponential(min=1,  
max=8))  
async def fetch_gstr2b(gstin: str, period: str) -> Gstr2bResponse:  
    async with httpx.AsyncClient(timeout=10.0) as client:  
        response = await client.post(  
            f"{settings.gsp_base_url}/gstr2b/all",  
            json={"gstin": gstin, "period": period},  
        )  
        response.raise_for_status()  
        return Gstr2bResponse.model_validate(response.json())
```

A failed external call after retries exhausted creates a task (e.g. “GSP fetch failed — retry manually”) rather than silently dropping the operation or crashing a background worker.

CG7 — Duplicate-prevention check before any Tally write

Verified in ADR 0001: Tally’s XML import does not prevent duplicate vouchers on its own. Any code path that posts a voucher to Tally must check for an existing match (vendor GSTIN + invoice number + date) before posting, and this check is not optional or deferrable — it ships in the same PR as the voucher-posting logic, not as a follow-up.

```
async def post_voucher(voucher: DraftVoucher) -> PostedVoucher:  
    existing = await find_existing_voucher(
```

```
        gstin=voucher.vendor_gstin,  
        invoice_number=voucher.invoice_number,  
        date=voucher.invoice_date,  
    )  
    if existing is not None:  
        raise DuplicateVoucherError(existing_id=existing.id)  
    return await tally_connector.post_voucher(voucher)
```

CG8 — Human-judgment paths go through the Task Engine, never ad hoc

Per ADR 0005: any agent exception or judgment call creates a Task via the shared Task Engine service. No agent module sends its own notification, writes its own “pending review” flag, or otherwise routes around the Task Engine — this is the platform’s core enforcement mechanism, and bypassing it in one module defeats the purpose for the whole system.

✗ Bad:

```
if confidence < 0.7:  
    send_email_to_staff("Please review this voucher")
```

✓ Good:

```
if confidence < CONFIDENCE_THRESHOLD:  
    await task_engine.create_task(  
        task_type="low_confidence_voucher_review",  
        client_id=voucher.client_id,  
        linked_record=voucher.id,  
        due_at=in_hours(12),  
    )
```

CG9 — Pydantic models for every request/response shape

No route handler accepts or returns a raw dict. Every request body and response shape is a Pydantic model, defined once and reused, not redefined per endpoint.

```
class VoucherApprovalRequest(BaseModel):  
    voucher_id: str  
    approved_by: str  
  
    @field_validator("voucher_id")  
    @classmethod  
    def must_be_valid_uuid(cls, v: str) -> str:  
        UUID(v) # raises ValueError if invalid  
        return v
```

CG10 — Testing: pytest, ≥85% coverage, all external services mocked

`pytest tests/ --cov=. --cov-report=term-missing --cov-fail-under=85`

Tally, GSP, and OCR calls are mocked at the client boundary in every test — CI never calls a real external service. The WhiteBooks and Tally sandboxes (per ADR 0002, ADR 0001) are for manual integration testing only, run separately from the automated suite, not wired into CI.

CG11 — Base product vs. practice customization (ADR 0010 extension-point pattern)

This resolves the open extension-point definition flagged in ADR 0010.

- Core modules (`services/api/agents/`, `services/api/task_engine/`, etc.) contain only generic, configuration-driven logic. They must never import from, or contain conditional logic branching on, a specific practice.
- Practice-specific customization lives under `services/api/practices/{practice_slug}/` and registers itself against defined extension points (a plugin-style registry, not direct monkey-patching of core modules).
- A core module may expose an extension point (e.g. `WorkingPaperTemplateProvider`), but never reaches into `practices/` directly — the dependency direction is one-way, practices depend on core, never the reverse.
- Before adding anything to `practices/{practice_slug}/`, check whether it's actually configuration (belongs in that practice's settings/service-catalog data, per CG4) rather than code — most “customizations” turn out to be config once examined properly.
- Code that starts in `practices/` and proves broadly useful gets promoted into core in its own PR, with the practice-specific version removed — not left duplicated indefinitely.

Logging Standard — Practice Automation Platform

v0.1

Changelog

Version	Date	Summary
v0.1	2026-08-16	Initial standard: structured logging with correlation fields, log-level definitions calibrated to platform events, the audit-trail boundary (compliance events don't go through the logger), and PII/masking rules given the financial data this platform handles.

1. Why this exists

The platform touches client GSTINs, financial amounts, and task/approval history for a compliance-regulated domain. Logging needs to be useful for debugging without becoming an accidental second, unstructured copy of sensitive data, and without being the system of record for actions that actually need a real audit trail (approvals, overrides, reassignments).

2. Structured logging, not `print()`

All logging goes through Python's `logging` module, configured to emit structured JSON (not plain text) so log lines are filterable by field once shipped to any log aggregation tool. `print()` is permitted only in one-off scripts under `scripts/`, never in `services/api/` route handlers, agents, or background workers.

```
import logging
```

```
logger = logging.getLogger(__name__)
```

```
logger.info(  
    "Voucher staged for review",  
    extra={"client_id": client_id, "voucher_id": voucher.id,  
    "confidence": confidence},  
)
```

3. Correlation fields — bind once, not per call

Every logger instance used inside an agent or service should be bound with a consistent set of correlation fields at construction, rather than repeating them on every log call:

Field	When present
practice_id	Always — even in a single-practice deployment, this keeps log format consistent for the base-product model (ADR 0010) where the same code runs across separate deployments
client_id	Any operation scoped to a specific client
task_id	Any operation that created or acted on a Task
agent	Which agent emitted the line (bookkeeping, reconciliation, email_intake, etc.)
request_id	Any log line originating inside an API request

```
def get_logger(agent: str, **bindings: str) -> logging.LoggerAdapter:  
    return logging.LoggerAdapter(logging.getLogger(agent), {"agent":  
agent, **bindings})  
  
logger = get_logger("reconciliation",  
practice_id=settings.practice_id, client_id=client.id)  
logger.info("GSTR-2B fetch completed", extra={"matched": 312,  
"mismatched": 4})
```

4. Log levels

Level	When to use	Example
debug	Off in production. Step-by-step tracing during development.	Raw OCR confidence scores per field before thresholding
info	Normal lifecycle events — the platform doing what it's supposed to do.	Voucher posted, reconciliation run completed, task created, GSP token refreshed
warn	Recoverable, but worth noticing.	External API retry triggered, OCR confidence below threshold (routed to task, not failed), stale Tally connection detected
error	An operation failed and a human or the system needs to know.	GSP call failed after retries exhausted, Tally voucher post rejected, OCR pipeline

Level	When to use	Example
		crashed on a document
Controlled by LOG_LEVEL: debug in development, info in production, silent (or a no-op logger, per §6) in tests by default.		

5. What must never go through the logger: compliance events

Per the platform's audit-trail requirement (PRD ID-05, and the DPDP accountability principle established in the PRD's architecture notes), the following are **not** logging events — they go through a dedicated `AuditTrailService` that writes an immutable, queryable record, not a log line that scrolls away:

- Voucher or invoice approval/rejection
- Task reassignment
- Any override of an agent's suggestion by a human
- Client record creation, edit, or deactivation
- Service catalog or pricing changes

 *Bad — this is a compliance event, not a log line*
`logger.info("Invoice approved", extra={"invoice_id": inv.id,
"approved_by": user.id})`

 *Good*
`await audit_trail.record(
event_type="invoice_approved",
actor_id=user.id,
subject_id=inv.id,
practice_id=practice_id,
)`

`AuditTrailService` is a separate, not-yet-built component — flagged here so route handlers are written against the right abstraction from the start rather than retrofitted later.

6. PII and financial data: what never appears in a log line

- **GSTIN**: log only the last 4 characters (*****1Z5), never the full value.
- **Bank account numbers**: never logged, masked or otherwise — if a bank reference is needed for debugging, log the internal record ID and look up the account separately.
- **Raw document content** (invoice text, extracted line items in full): never logged. Log the document ID and a summary ("3 line items extracted, confidence 0.91"), not the content itself.
- **Client contact details** (phone, email): avoid in log lines where an internal ID would do the same job.

This isn't about hiding information from the team — it's about not creating a second, less-controlled copy of exactly the data the platform's thin-layer architecture (ADR 0004) was designed to avoid duplicating.

7. Testing

Inject a no-op or in-memory logger in tests rather than asserting against real log output, unless the test is specifically about logging behavior itself.

```
class NullLogAdapter(logging.LoggerAdapter):
    def process(self, msg, kwargs):
        return msg, kwargs

def test_voucher_posting_does_not_crash_on_missing_ledger() -> None:
    sut =
    BookkeepingAgent(logger=NullLogAdapter(logging.getLogger("test"), {}))
    ...
```

If a test needs to assert a specific log call happened (e.g. confirming a warning fires on low confidence), inject a mock logger and assert on it directly rather than capturing stdout.

Testing Strategy — Practice Automation Platform

v0.1

Changelog

Version	Date	Summary
v0.1	2026-08-16	Initial strategy: test pyramid, external-service mocking approach, critical end-to-end flows, test data rules (DPDP-aligned), confidence-threshold boundary testing, and CI gates.

1. Test pyramid

/\	
/e2e\	A handful – the critical flows in §3, run nightly
/-----\	
/ integr. \	Against sandboxes (Tally API Explorer, WhiteBooks
GSP sandbox) – run on merge to main, not every PR	
/-----\	
/ unit tests \	The bulk – every agent's matching/extraction
logic, run on every PR	
/-----\	

Most of the platform’s actual risk lives in unit-testable logic — fuzzy vendor matching, confidence scoring, duplicate detection, Decimal-precision reconciliation math — not in the plumbing around it. Weight the suite accordingly rather than defaulting to broad, shallow end-to-end coverage.

2. What gets mocked, and what doesn’t

Per CG10 (Coding Guidelines): Tally, the GSP, and OCR calls are mocked at the client boundary in the unit/PR-level suite. CI never calls a real external service on every PR — those are rate-limited, sometimes cost money per call (GSP API), and would make the fast feedback loop slow and flaky.

- **Unit tests:** mock everything external. Fast, run on every push.
- **Integration tests:** run against the real sandboxes — TallyPrime API Explorer (per ADR 0001) and the WhiteBooks GSP sandbox (per ADR 0002) — on merge to main, not on every PR. These catch drift between what the mocks assume and what the real APIs actually do.

- **Manual/exploratory testing:** anything requiring a real client's live Tally company or live GSTIN never happens in an automated test — only in a controlled staging walkthrough with the practice's explicit involvement.
-

3. Critical end-to-end flows (test these explicitly, not incidentally)

These mirror the platform's actual value proposition — if these silently break, the automation isn't working even if unit tests are green.

1. **Document intake — bookkeeping — voucher posted:** a sample invoice enters via the Dropbox webhook, gets OCR'd, matched to a vendor ledger, staged, approved, and posted — verify the posted voucher matches the source document's amounts exactly (Decimal precision, per CG5).
 2. **GSTR-2B fetch — reconciliation — task created:** a known mismatch in fixture data produces exactly one task, with the correct reason code and variance amount — not zero tasks (silently missed) and not duplicate tasks (per CG7's duplicate-prevention principle, applied to reconciliation flagging too).
 3. **Task overdue — escalation:** a task past its due date auto-escalates to the correct role per the routing rule table (TE-02) — this is the mechanism the whole “not another notification system” design (ADR 0005) depends on, so it needs a real test, not just a code review nod.
 4. **Invoice draft — approval — GST-compliant output:** a generated invoice contains a valid GSTIN, correct HSN/SAC codes, and a CGST/SGST breakup that sums correctly to the total (CB-09) — malformed invoices are a compliance problem, not just a bug.
-

4. Test data: synthetic only, never real client documents

No real client's GSTIN, financial documents, or extracted data ever appears in a test fixture, committed to the repository, or used in CI — this follows directly from the Logging Standard's PII rules (§6) and the platform's data-minimization architecture (ADR 0004). Test fixtures are synthetic: invented company names, valid-format-but-fake GSTINs, plausible but fictional invoice amounts.

```
# tests/fixtures/invoices.py
SAMPLE_INVOICE = {
    "vendor_name": "Coastal Test Traders",
    "vendor_gstin": "33AAAAA0000A1Z5", # valid checksum format, not a
real GSTIN
    "invoice_number": "TEST-INV-0001",
    "amount": "18400.00",
}
```

5. Confidence-threshold boundary testing

Every agent that uses a confidence score to decide auto-stage vs. flag-for-review (BK-05) needs explicit tests at the boundary, not just “obviously high” and “obviously low” cases — boundary behavior is where real bugs hide.

```
def
test_voucher_at_exact_confidence_threshold_is_flagged_not_auto_staged(
) -> None:
    result = score_and_route(confidence=CONFIDENCE_THRESHOLD) #
exactly at the line
    assert result.status == "flagged_for_review" # threshold is
inclusive on the safe side
```

6. CI gates

Required to pass before merge:

1. `ruff check . --no-fix` (CG1)
2. Type check (mypy or ruff's type-aware rules)
3. Unit test suite with coverage $\geq 85\%$ (CG10)
4. No new dependency without a pinned version (CG9-equivalent in the eventual `requirements.txt` discipline)

Run on merge to main, not blocking every PR:

5. Integration suite against Tally API Explorer and WhiteBooks sandbox
 6. The 4 critical end-to-end flows in \$3
-

7. What this doesn't cover

Load/performance testing has its own document (`audit-platform-performance-scaling-v0.1.md`) — this strategy covers correctness, not throughput or latency targets.

Security Standard — Practice Automation Platform

v0.1

Changelog

Version	Date	Summary
v0.1	2026-08-16	Initial standard: authentication/session rules, role-based authorization enforcement, secrets handling, encryption, audit-trail immutability, input validation, dependency scanning, and GSP-call rate limiting.

1. Authentication & session handling

Per ID-01/ID-04 (PRD): role-based login (Proprietor / Senior / Junior), with session handling appropriate for shared office devices.

- Session tokens expire after a fixed idle period (default 8 hours) — no indefinitely-lived sessions, since office devices are often shared.
- No credentials or session tokens persisted in a way that survives a browser restart on a shared machine beyond the session's own expiry.
- Failed login attempts are rate-limited per account to prevent credential-stuffing against the small, known staff user base.

2. Authorization is enforced server-side, not just hidden in the UI

Every API endpoint checks the caller's role against what the action requires — a junior staff member's browser simply not showing an “Approve invoice” button is not authorization; the endpoint itself must reject the request if called directly.

```
async def approve_invoice(invoice_id: str, current_user: User =  
    Depends(get_current_user)) -> None:  
    if current_user.role not in {"proprietor"}:  
        raise HTTPException(status_code=403, detail="Invoice approval  
requires proprietor role")  
    ...
```

This matters more than usual here because the routing rules (TE-02) explicitly assign some actions — invoice approval, client communication approval — to the proprietor only; a UI-only restriction would silently violate that design the moment anyone calls the API directly.

3. Secrets: storage and rotation

Storage follows CG4 (Coding Guidelines) — centralized settings, never committed to git. In addition:

- Tally connector credentials and GSP client secret are stored in a secrets manager appropriate to the hosting environment (e.g. AWS Secrets Manager for the AWS Mumbai deployment per ADR 0007/0009), not as plain environment variables in a deployment config file.
 - Credential rotation: GSP and Tally credentials are rotated on a defined schedule (recommend every 90 days) and immediately if any compromise is suspected — this needs an actual runbook step in the Dev Readiness Checklist, not just a policy statement.
 - No credential ever appears in a log line (per the Logging Standard §6) or in an error message returned to the frontend.
-

4. Encryption

- **In transit:** TLS everywhere — the PWA to the API, the API to Tally, the API to the GSP, the API to Dropbox. No plaintext HTTP anywhere in the request path, including internal service-to-service calls if the deployment architecture ends up with more than one service.
 - **At rest:** the database is encrypted at rest (standard for AWS RDS/managed Postgres — this is a configuration checkbox, not custom engineering, but it needs to be explicitly verified during deployment setup, not assumed).
 - Client documents that transit through the platform (per ADR 0004's thin-layer principle) are not stored at rest by the platform itself beyond what's needed for active processing — they live in Dropbox as the system of record.
-

5. Audit trail is append-only and separate from application data

Per the Logging Standard §5: compliance events (approvals, overrides, reassignments, client record changes) go through `AuditTrailService`, not the logger. That service's storage must be append-only — no UPDATE or DELETE on audit records through the normal application data path. If an audit record needs correction, that's a new record referencing the original, not a mutation of history.

6. Input validation beyond Pydantic

CG9 requires Pydantic models for every request/response shape, which handles shape and type validation. Beyond that:

- Any value that flows into a Tally XML payload or a GSP API call is validated against its expected format (GSTIN checksum, invoice number character set) before being sent — a malformed or malicious value reaching an external system is a bigger problem than one rejected at the platform's own boundary.

- File uploads (documents arriving via Dropbox/email) are validated for expected file type before OCR processing — the intake pipeline should reject an unexpected executable or script disguised with a document extension, not attempt to process it.
-

7. Dependency management

- All dependencies pinned to exact versions (mirrors the discipline in CGI, applied to the dependency file itself, not just linting).
 - Automated vulnerability scanning on the dependency tree (e.g. `pip-audit` or GitHub's Dependabot alerts) runs on a schedule, not just at initial setup — new CVEs get discovered in already-shipped dependencies constantly.
-

8. GSP call rate limiting — a cost-safety concern, not just a security one

WhiteBooks' (or any chosen GSP's) API is billed per call or per plan tier (per ADR 0002/RC-01b). A bug that causes runaway repeated calls — a retry loop without a cap, a webhook triggering duplicate fetches — isn't just a performance problem, it's a real cost problem given the volume this platform is meant to reach (500 returns/month). Internal rate limiting per client per period should exist independent of whatever limit the GSP itself enforces, as a defense against the platform's own bugs, not just external abuse.

9. Incident response basics

- If a credential is suspected compromised: rotate it immediately (per §3), and treat any data accessed with it as potentially exposed until reviewed.
- If client financial data is suspected exposed: this is a DPDP-relevant event — routes to whoever owns legal/compliance for the practice (per the PRD's still-open DPDP legal item), not just an engineering fix-and-move-on.
- A minimal incident log (what happened, when noticed, what was done) should exist even before a formal incident-response runbook is written — the Dev Readiness Checklist should include creating this runbook as an explicit task.

Performance & Scaling — Practice Automation Platform

v0.1

Changelog

Version	Date	Summary
v0.1	2026-08-16	Initial targets: current-scale assumptions, API/OCR/queue performance targets, database indexing for reconciliation matching, and concrete scaling triggers — all scoped per-deployment, per ADR 0009’s single-tenant model.

1. Scale assumptions (per deployment)

Per ADR 0009, every deployment is single-tenant — these targets are **per practice**, not aggregated across future customers. A second practice’s deployment gets its own instance and its own performance profile, not a shared pool that needs to account for combined load.

For this first deployment specifically: - ~100 clients today, target ~500 within 6 months (per the proposal’s central goal) - ~10 concurrent staff users at peak - Document volume: moderate — a few thousand documents/month at 500-client scale, not industrial batch volume

These numbers matter because they justify *not* over-engineering for scale that doesn’t exist yet — CPU-only OCR (ADR 0008) and a lightweight hosting footprint (ADR 0007/0009) are correct choices at this scale, not premature economizing.

2. API response time targets

Endpoint class	Target (P95)	Rationale
Dashboard/task-list reads	< 500ms	Staff check this frequently through the day; sluggish reads erode trust in the tool
Voucher/reconciliation review actions (approve, reject)	< 1s	Needs to feel responsive during active review sessions
Document upload — OCR trigger acknowledgment	< 2s (the OCR itself runs async, per ADR 0003’s queue pattern)	The user-facing action is “acknowledged,” not “OCR complete”
Report/working paper	< 5s	Less frequent, less latency-

Endpoint class	Target (P95)	Rationale
generation		sensitive action

These are targets for a lightly-loaded single-deployment instance, not aspirational figures for a system under heavy concurrent load that doesn't exist at this scale.

3. OCR throughput

Per ADR 0008: PaddleOCR runs CPU-first, since processing is async (queue-based, per ADR 0003) with no real-time latency requirement.

- **Target:** process a typical invoice document within 30 seconds of being picked up from the queue, on CPU.
- **Scaling trigger:** if the document processing queue's backlog regularly exceeds a few hundred pending items (i.e., processing can't keep pace with intake even during normal hours, not just a temporary morning spike), that's the signal to provision GPU — not before. Don't provision GPU based on projected volume; provision it based on observed queue backlog.

4. Background job / queue sizing

Celery + Redis (per ADR 0007) handles: Dropbox webhook processing, OCR jobs, GSP API calls, email intake processing.

- Start with a small worker pool (2-4 workers) sized for the current document volume — this is not a number to over-provision for hypothetical future load, per the same reasoning as §3.
- Queue depth is a monitored signal (ties to the still-open observability tooling decision) — sustained queue growth over time, not point-in-time spikes, is what triggers adding worker capacity.

5. Database performance — reconciliation matching is the hot path

The Reconciliation Agent's core operation (RC-02: match purchase register line items against GSTR-2A/2B by GSTIN + invoice number + amount) is the platform's most query-intensive operation at scale, since it runs across potentially thousands of line items per client per period.

- Index (`client_id`, `vendor_gstin`, `invoice_number`) on the purchase register / voucher table — this is the primary lookup path for matching and for the duplicate-prevention check (CG7).
- Avoid N+1 query patterns in the matching logic — batch-fetch GSTR-2B line items for a period once, match in memory or via a single set-based query, not one query per purchase register line.

- Revisit indexing once real volume from Phase 0/1 usage is available — designing the “correct” index set against assumed access patterns before real query logs exist is guesswork; this is a starting point, not a final answer.
-

6. PWA load performance

Per ADR 0006 (desktop-first PWA):

- Initial load target: interactive within 2-3 seconds on a typical office broadband connection — this is a desk-based tool, not a mobile app optimized for constrained networks, so the bar is “doesn’t feel slow,” not “works on 2G.”
 - No specific bundle-size budget is set yet — revisit once the frontend build exists and a real baseline can be measured, rather than guessing a number now.
-

7. Scaling triggers — a summary table

Signal	Action
OCR queue backlog sustained > few hundred items	Provision GPU for PaddleOCR
Celery queue depth trending up over days, not just spiking	Add worker capacity
API P95 response time exceeding targets in \$2 under normal (not peak) load	Investigate query performance before scaling hardware
A second practice signs on	New deployment (per ADR 0009) — not a capacity question for this instance at all

8. What this doesn’t cover

Formal load testing tooling and process aren’t specified yet — this document sets targets; the Dev Readiness Checklist should include setting up actual load-testing tooling as a task, once there’s a working system to test against rather than a specification to guess load-test scenarios for.

Sprint Plan — Practice Automation Platform

12 Sprints × 2 Weeks = 24 Weeks (+ Phase 0 Baseline) | v0.1

Changelog

Version	Date	Summary
v0.1	2026-08-16	Initial plan: Phase 0 baseline + 12 sprints, sequencing the Feature Backlog's 102 features against the module dependency order established across the ADRs. Each sprint lists goal, ADRs exercised, and deliverables traced to requirement IDs.

Sequencing logic

Sprints follow the same dependency order the ADRs and Feature Backlog already established: foundation before agents (a task can't route anywhere without the Task Engine; a voucher can't post without the Tally Connector), Bookkeeping before Reconciliation (the majority-path data has to exist in Tally before it can be reconciled), and Billing after enough agents exist to generate real billable events. The customer's own two highest-priority pain points — bookkeeping volume and client follow-up — land in Sprints 4–6, not at the end.

Phase 0 — Baseline (Weeks -3 to 0, pre-sprint)

Not a build sprint — a measurement and validation phase before any application code is written, per the original proposal's Phase 0.

- Use existing task-sheet data to measure real time-per-task (P0-01)
- Confirm Tally Cloud access method with the provider — port 9000 reachability (P0-02)
- Map shared inbox (work@xyz.com) routing logic to clients/staff (P0-03)
- WhiteBooks GSP sandbox spike — validate GSTR-2B fetch + GSTR-1/3B filing endpoints (P0-04)
- PaddleOCR accuracy validation against 20–50 real sample invoices (P0-05)
- IaC tooling decision made (Terraform, per the still-open ADR item) — this needs resolving before Sprint 1 begins, since Sprint 1 provisions infrastructure

Sprint 1-2 (Weeks 1-4) — Foundation

Sprint goal: Repo scaffolded, infrastructure provisioned, auth working, Task Engine operational, Tally Connector reading real data.

ADRs exercised: 0001 (Tally XML-HTTP), 0004 (thin-layer architecture), 0005 (Task Engine), 0007 (Python/FastAPI stack), 0009 (deployment model), 0010 (extension-point scaffold)

Deliverables: - Repo scaffolded per Coding Guidelines: ruff config, pytest, CI pipeline (Testing Strategy §6) - Infrastructure provisioned via IaC on AWS ap-south-1 (ADR 0007/0009) - practices/{practice_slug}/ extension-point directory scaffolded, empty but structurally in place (CG11) - Postgres schema for core-platform entities: Client, User, Task, Document, ExtractedData, TallyCompany, Voucher, VendorMapping, EmailSenderMap (per the core ER diagram) - Auth: JWT login/logout, role-based access control enforced server-side (ID-01 to ID-05, Security Standard §1-2) - Secrets management wired to AWS Secrets Manager (Security Standard §3) - Task Engine: creation, routing rules, status lifecycle, auto-escalation (TE-01 to TE-09) - Tally Connector: XML-over-HTTP connection, tested against the TallyPrime API Explorer sandbox first, then the practice's real Tally Cloud (TC-01 to TC-05) - Dashboard API + Dashboard screen wired to real Task Engine data (PM-01)

Sprint 3 (Weeks 5-6) — Intake Pipeline

Sprint goal: Documents flow automatically from email and Dropbox into the platform, with OCR extraction running.

ADRs exercised: 0003 (Dropbox webhooks + async queue), 0008 (PaddleOCR)

Deliverables: - Email Intake Agent: shared inbox monitoring, sender-to-client mapping, auto-file to Dropbox (EI-01 to EI-06) - Document Intake: Dropbox webhook + list_folder/continue pattern, async queue processing respecting the 10-second response constraint (DI-01, DI-01a, DI-01b) - PaddleOCR (PP-StructureV3) pipeline live, validated against the Phase 0 sample set (DI-02, DI-03) - Document checklist tracking, missing-item flagging (DI-04, DI-05) - My Tasks screen wired to real data (TE-07)

Sprint 4-5 (Weeks 7-10) — Bookkeeping Agent

Sprint goal: The highest-leverage agent, per the customer's own stated priority — documents become staged, reviewable, postable vouchers.

ADRs exercised: CG5 (Decimal precision), CG7 (duplicate-prevention)

Deliverables: - Line-item extraction from OCR output — vendor, date, amount, GST, HSN (BK-01) - Fuzzy vendor-to-ledger matching engine (BK-02) - New-ledger-creation task flow — flagged, not auto-created (BK-03) - Draft voucher generation with correct tax split (BK-04) - Confidence scoring, auto-stage vs. flag logic (BK-05) - Bookkeeping Review screen wired to real staged vouchers, document preview alongside (BK-06) - Voucher posting to Tally via Connector, with the duplicate-prevention check running before every post (BK-07, BK-08) - Client Workspace screen wired to real per-client module status

Sprint 6 (Weeks 11-12) — Client Communication

Sprint goal: Follow-up automation live — the practice's stated #1 pain point.

Deliverables: - Template library + scheduled auto-send for routine reminders (CC-01, CC-02) - Judgment-required communication drafting + proprietor approval queue (CC-03) - Communication log (CC-04) - Priority flagging for time-sensitive, client-dependent actions like OTP submission (CC-05) - Client Communication screen wired to real data (approval queue + log)

Sprint 7 (Weeks 13-14) — Reconciliation Agent

Sprint goal: GSTR-2A/2B reconciliation running through a direct GSP integration, with Winman removed from the critical path.

ADRs exercised: 0002 (GSP integration, bypassing Winman)

Deliverables: - GSP integration (WhiteBooks or the vendor confirmed during Phase 0's sandbox spike) — GSTR-2B fetch (RC-01) - Matching engine: GSTIN + invoice number + amount (RC-02) - Auto-confirm for matched items, ITC claim summary (RC-03, RC-05) - Mismatch queue with reason codes, routed as tasks (RC-04) - Reconciliation Review screen wired to real data

Sprint 8 (Weeks 15-16) — Validation/Compliance + TDS/TCS

Sprint goal: Pre-filing checks and TDS reconciliation, reusing the matching patterns built in Sprint 7.

Deliverables: - GSTR-1 vs. 3B, HSN code, tax rate, and TDS-vs-26AS checks (VC-01 to VC-05) - Per-client filing-readiness status (VC-06) - TDS deduction-vs-deposit matching (TDS-01 to TDS-03) - Draft quarterly TDS return generation (TDS-04)

Sprint 9 (Weeks 17-18) — Bank Reconciliation + Working Paper

Sprint goal: Round out the compliance layer; auto-assemble the factual backbone of every engagement's working paper.

Deliverables: - Bank account/transaction management, auto-matching by date/amount/reference (BR-01 to BR-04) - Working paper auto-population from Reconciliation and Validation output (WP-01, WP-02) - Judgment-notes field + finalization workflow, filed to Dropbox (WP-03, WP-04) - Reports & Working Papers screen wired to real data

Sprint 10 (Weeks 19-20) — Client Profiling & Billing

Sprint goal: Billing automation, built from scratch since no rate card exists today.

Deliverables: - Client master profile CRUD, add/edit/deactivate (CB-01) - Service catalog + categories, reflecting the practice's actual service lines (CB-02) - Per-client price overrides with recorded reason/approval (CB-03) - Billing-trigger mapping from task/agent completion events (CB-04) - Draft invoice generation, GST-compliant (GSTIN, HSN/SAC, tax breakup) (CB-05, CB-09) - Invoice approval flow (CB-06) - Invoice history (CB-07) - Admin — Clients and Service Catalog screens wired to real data; Billing screen wired

Sprint 11 (Weeks 21-22) — Practice Management + Admin Completion

Sprint goal: Give the proprietor full practice-wide visibility; close out the remaining Admin screens.

Deliverables: - Staff workload view, capacity indicator, deadline calendar (PM-02 to PM-04) - Time-per-task tracking, extending the practice's existing task-sheet habit (PM-05) - Outstanding invoice visibility on the dashboard (PM-06) - Admin — Assignment Rules screen wired to the real routing rule table (TE-02) - Admin — Connections screen wired to real Tally/GSP connection health (TC-05, TC-08) - Tier reassignment workflow (CB-08)

Sprint 12 (Weeks 23-24) — Hardening & Launch Preparation

Sprint goal: Production-ready. Every critical flow verified end-to-end, security and performance validated against their respective standards, not just assumed.

Deliverables: - All 4 critical end-to-end flows tested against staging (Testing Strategy §3) - Security review against the Security Standard checklist — auth, secrets, encryption, audit-trail immutability, GSP call rate limiting - Performance validated against targets in Performance & Scaling §2 and §5, under realistic (not synthetic) data volume - Credential-rotation runbook written (Security Standard §3, still-pending task) - Load-testing tooling set up (Performance & Scaling §8, still-pending task) - DPDP legal review checkpoint — routed to the practice's legal/compliance contact, not resolved by engineering (still-open item) - ADR review: confirm all 10 ADRs still reflect what was actually built; update any that drifted - Staging — production deployment on AWS ap-south-1

Summary

Sprint	Weeks	Focus	Key modules
Phase 0	-3-0	Baseline measurement	—
1-2	1-4	Foundation	Identity, Task Engine, Tally

Sprint	Weeks	Focus	Key modules
			Connector
3	5–6	Intake pipeline	Email Intake, Document Intake
4–5	7–10	Bookkeeping	Bookkeeping Agent
6	11–12	Follow-up automation	Client Communication
7	13–14	GST reconciliation	Reconciliation Agent
8	15–16	Compliance checks	Validation, TDS/TCS
9	17–18	Remaining compliance	Bank Reconciliation, Working Paper
10	19–20	Billing	Client Profiling & Billing
11	21–22	Practice visibility	Practice Management, Admin
12	23–24	Hardening	Security, performance, launch prep

Total: ~27 weeks including Phase 0 — comparable to COCA’s 30-week plan despite a smaller module count, reflecting a smaller assumed team rather than a shorter task list.